VEEP

Verifiable Enterprise Experience Platform

Knowledge Graph: Design and Deployment

A complete architecture, data model, and rollout plan for the entity-relationship graph that underlies candidate evidence, trust scoring, and employer discovery across the VEEP platform.

PREPARED FOR: FOUNDERS · TECHNICAL CO-FOUNDERS · ENGINEERING LEADERSHIP

UK · CANADA · AUSTRALIA

DOCUMENT CONTROL

Confidentiality & Document Control

DISTRIBUTION

- Founders and technical co-founders, PHEXARA LTD
- Engineering leadership: Developer, AI Engineer, Data Engineer
- Cybersecurity Analyst (security and compliance review)
- Investors and incubator technical reviewers, on request only

CLASSIFICATION

This document is classified CONFIDENTIAL under PHEXARA LTD information handling policy. It contains system architecture, data model, and security design detail for the VEEP platform and must not be shared outside the distribution list above without written approval.

DOCUMENT DETAIL

Document title	VEEP Knowledge Graph — Design & Deployment
Prepared by	PHEXARA LTD — Engineering & AI
Version	1.0
Status	For engineering review
Related documents	VEEP Workflow Analysis; VEEP Plain-English Report; PHEXARA Brand Strategy Portfolio v2
Companion source	VEEP Master Blueprint — Expanded Edition (Amended)

CONTENTS

Table of Contents — Part I

01 Strategic Rationale 8

02 Ontology & Entity Model 20

03 Relationships & Graph Patterns 40

04 Hybrid Search & AI Integration 55

05 Technology Architecture 65

CONTENTS

Table of Contents — Part II

06 **Build & Deployment Plan** 110

07 **Operations & Governance** 130

08 **Roadmap, Team & Investment** 139

09 **Close** 144

APPENDICES REFERENCED THROUGHOUT

- Full 14-node property schema reference — consolidated in Section 02
- Sample query library (GQL, openCypher-compatible patterns) — consolidated in Section 03
- Risk register and RACI — consolidated in Sections 07–08

READING GUIDE

How to Read This Deck

Every diagram in this deck uses one consistent colour language for actors and one consistent shape language for graph structure. Both are carried directly from the VEEP product and the PHEXARA visual identity system.



Candidate

The person building a verified profile



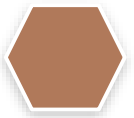
Employer

The organisation discovering and hiring talent



AI Persona

AI manager, colleague, or evaluator persona



System

Automated platform / infrastructure process

GRAPH SHAPE LANGUAGE



Hexagon = a node (an entity in the graph)

Line = a relationship (an edge in the graph), labelled with its verb

PRIORITY CODE

P0 Launch-blocking

P1 High priority

P2 Enhancement

EXECUTIVE SUMMARY

Why VEEP Is Building a Knowledge Graph

VEEP's product promise is verifiable, evidence-backed professional experience. That promise depends on connecting many small facts — a task, an artefact, a score, a message, a hash — into one provable chain that ends in a trust score an employer can rely on. A knowledge graph is the data structure built for exactly that job: entities as nodes, relationships as edges, and every claim traceable to its evidence.

14

Node types

modelled across candidates, work, and evidence

20+

Edge types

connecting actors, work objects, and trust

5

Deployment phases

aligned to VEEP's 7-month build plan

3

Launch regions

UK, Canada, Australia — one shared graph

WHAT THIS DECK DELIVERS

- The complete entity-relationship ontology behind candidate, employer, task, artefact, score, and trust data
- A selected graph database technology and a full target-state architecture on GCP
- A phased build and deployment plan mapped directly to VEEP's existing 5-stage roadmap
- Operating, monitoring, and governance procedures for running the graph in production

Scope, Sources & Method

This deck focuses exclusively on the design and deployment of a knowledge graph layer for VEEP: how candidate, employer, work, evidence, and trust data should be modelled as a graph, and how that graph should be built, deployed, and operated on VEEP's existing GCP infrastructure.

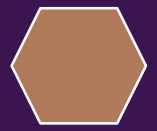
IN SCOPE

- Full graph ontology: 14 node types, 20+ relationship types
- Graph database technology selection and target architecture
- Ingestion pipeline from existing AlloyDB for PostgreSQL and Cloud Storage
- Hybrid graph + vector retrieval for matching and evaluation
- Phased build, deployment, and operations plan

OUT OF SCOPE

- Full 15-role task catalogue detail — see VEEP Workflow Analysis
- Complete financial model and unit economics
- Brand visual identity system — see Brand Strategy Portfolio v2
- Front-end UI implementation detail beyond illustrative mock-ups

SOURCE DOCUMENTS *VEEP Workflow Analysis · VEEP Plain-English Report · PHEXARA Brand Strategy Portfolio v2 · VEEP Master Blueprint (Amended)*



Strategic Rationale

Why VEEP's trust model needs a graph-native data structure, and what that structure must achieve.

STRATEGIC RATIONALE

The Trust Problem VEEP Must Solve

VEEP's verified profile is only as trustworthy as the chain of evidence behind it. Every score on a profile must be traceable back to a specific task, a specific submitted artefact, and a specific evaluation event. If that chain cannot be followed and proven end to end, the trust score is an assertion, not evidence — and VEEP's entire product promise depends on the difference.



Each arrow is a relationship that must be provable in the graph — not merely implied by row order in a table.

This chain, and every fact that feeds it, is what the knowledge graph exists to hold, connect, and make queryable.

STRATEGIC RATIONALE

Why Relational Storage Alone Is Insufficient

VEEP's AlloyDB for PostgreSQL database is correct for what it does: transactional storage of tasks, scores, and profile state. It is not built to answer questions that require walking several relationships at once, and VEEP's core trust and discovery questions are exactly that kind of question.

CAPABILITY	RELATIONAL ONLY (ALLOYDB FOR POSTGRESQL)	KNOWLEDGE GRAPH
Multi-hop traversal	Requires a chain of JOINs, one per hop, that gets slower and harder to write as the chain lengthens	Walks any number of relationships in one traversal, at consistent cost per hop
Schema flexibility	New relationship types require new join tables and migrations	New relationship types are added as new edges, without restructuring existing data
Evidence provenance	Provenance must be reconstructed by joining audit tables after the fact	Provenance is a first-class edge, queryable directly from any node
Similarity + structure together	Vector similarity (pgvector) and relationship structure live in separate query paths	Structure and similarity can be combined in a single hybrid traversal

STRATEGIC RATIONALE

Current Data Architecture: As-Is

The knowledge graph is a new layer added alongside the existing stack — it does not replace any system already in production.

Candidate App & Employer Portal

Next.js 14 + Tailwind

Auth Service

Google Identity Platform

Primary Database

AlloyDB for PostgreSQL
+ pgvector

Task Engine

Node.js + BullMQ on
Memorystore for Redis

Evidence Store

Google Cloud Storage
(hashed artefacts)

AI Personas

Claude API (Sonnet)
+ GPT-4o fallback

What is missing: a layer that treats relationships between these systems' data as queryable structure, not just foreign keys.

STRATEGIC RATIONALE

Five Multi-Hop Questions the Business Must Answer

1. Which candidates have verified evidence, above a trust score of 80, for the Cybersecurity Analyst role, in the UK, submitted in the last 90 days?
2. For a given trust score, which specific artefacts, evaluations, and AI persona interactions contributed to it, and in what proportion?
3. Which AI evaluator persona has been involved in an unusual concentration of high scores, across which roles and regions?
4. Which employers discovered a given candidate, through which search path, and did that path lead to a hire?
5. If an artefact's stored hash is challenged, what is the complete, provable chain from submission to verification?

Each question requires walking three to five relationships at once — the defining case for a graph.

Query Complexity: SQL Joins vs Graph Traversal

Same question — "trace the complete evidence chain behind one candidate's trust score" — expressed both ways.

RELATIONAL (SQL)

```
SELECT t.trust_score, s.score_value, a.artefact_hash,
  ev.evaluator_id, tk.task_id
FROM trust_scores t
JOIN scores s ON s.candidate_id = t.candidate_id
JOIN artefacts a ON a.score_id = s.id
JOIN evaluations ev ON ev.artefact_id = a.id
JOIN tasks tk ON tk.id = a.task_id
WHERE t.candidate_id = :id
  AND t.computed_at = (
    SELECT MAX(computed_at) FROM trust_scores
    WHERE candidate_id = :id);

-- 5 joined tables, 1 correlated subquery,
-- re-written for every additional hop
```

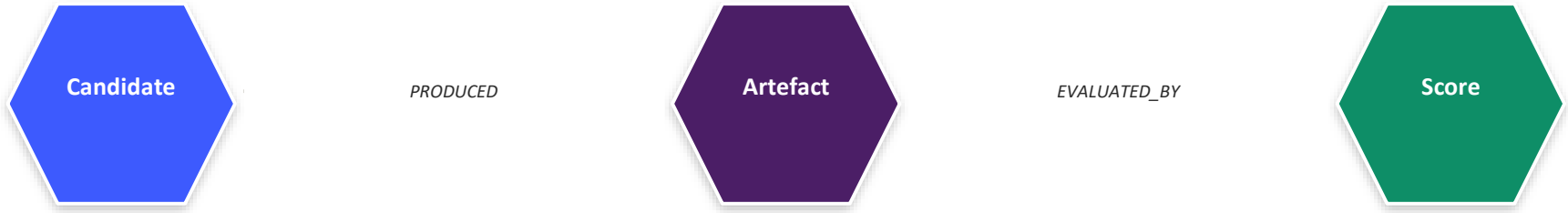
GRAPH (OPENCYPHER)

```
MATCH (c:Candidate {id: $id})
      -[:PRODUCED]->(a:Artefact)
      -[:EVALUATED_BY]->(sc:Score)
      -[:CONTRIBUTES_TO]->(t:TrustScore)
OPTIONAL MATCH (a)<-[:GENERATED]-(tk:Task)
OPTIONAL MATCH (sc)<-[:SCORED]-(p:AIPersona)
RETURN c, a, sc, t, tk, p;

// 1 traversal pattern, reads as the
// evidence chain itself, extends by
// adding one more relationship clause
```

What a Knowledge Graph Is, Precisely

A knowledge graph is a database in which data is stored as nodes (entities) and edges (relationships between entities), where both nodes and edges can carry properties, and where queries are expressed as traversals across those relationships rather than as joins across tables.



Node

An entity — a Candidate, an Artefact, a Score. Carries properties such as id, timestamp, and status.

Edge

A relationship between two nodes, given a direction and a verb, such as PRODUCED or EVALUATED_BY.

Property

A named attribute on a node or edge, such as trust_score_value or submitted_at.

Traversal

A query that walks a path of edges from a starting node to answer a question.

Knowledge Graph vs Vector Search vs Relational: Capability Matrix

VEEP needs all three. This matrix sets out which capability each one owns, so the architecture in Section 05 assigns work correctly.

Capability	Relational (AlloyDB for PostgreSQL)	Vector (pgvector)	Graph (Knowledge Graph)
Transactional writes (create task, submit artefact)	Strong	—	Weak
Semantic similarity (candidate ↔ employer matching)	—	Strong	Moderate
Multi-hop relationship traversal	Weak	—	Strong
Evidence provenance and audit trails	Moderate	—	Strong
Aggregation and reporting (dashboards, KPIs)	Strong	—	Moderate
Explainable trust-score reasoning	Weak	Weak	Strong

STRATEGIC RATIONALE

Design Goals for the VEEP Knowledge Graph

1 Provable evidence chains

Every score and trust value must resolve to a traversable path ending in a stored, hashed artefact.

2 Sub-second discovery queries

Employer search and filter operations must return in under one second at production scale.

3 Additive schema evolution

New node and relationship types must be addable without downtime or a breaking migration.

4 Regional data segregation

UK, Canada, and Australia data must be logically separable to meet regional compliance obligations.

5 Hybrid retrieval

Graph traversal and vector similarity must be combinable in a single query path for matching and evaluation.

6 Operable by a small team

The graph must be maintainable by VEEP's existing Data Engineer and AI Engineer without a dedicated graph specialist.

STRATEGIC RATIONALE

Success Metrics & KPIs for the Knowledge Graph Programme

METRIC	TARGET	MEASURED VIA
p95 evidence-chain traversal latency	**< 400 ms**	Query API
p95 employer discovery query latency	**< 800 ms**	GraphQL Gateway
Graph write lag behind source of truth	**< 5 seconds**	CDC pipeline metrics
Evidence chains resolvable end to end	**100%**	Data quality audit
Schema changes deployed without downtime	**100%**	CI/CD deployment log
Monthly graph infrastructure cost, at launch scale	**< £600**	Google Cloud Billing

Guiding Principles

1

Evidence, not assertion

No node exists in the graph without a source event; no edge exists without a cause. If it is in the graph, it happened.

2

Trust is a computed property, not a stored opinion

Trust scores are always derived by traversal from underlying evidence, never hand-set or hand-edited.

3

The graph augments, it does not replace

AlloyDB for PostgreSQL remains the transactional system of record; the graph is a derived, queryable view of relationships.

4

Additive by default

Schema changes add nodes, edges, or properties; they do not rename or remove without a governed migration, per Section 06.

5

Regional boundaries are structural

UK, Canada, and Australia data is partitioned in the graph design itself, not bolted on with query-time filters.

STRATEGIC RATIONALE

Programme Scope Boundary

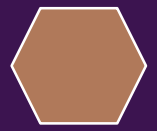
What the knowledge graph programme delivers, and what it deliberately leaves to existing or future workstreams.

DELIVERED BY THIS PROGRAMME

- The 14-node, 20+ edge ontology defined in Section 02
- Google Cloud Spanner Graph deployment, described in Section 05
- CDC ingestion from AlloyDB for PostgreSQL and Cloud Storage
- GraphQL query API for employer discovery and evidence lookup
- Phased rollout aligned to the existing 5-stage build plan

DELIBERATELY NOT IN THIS PROGRAMME

- Replacing AlloyDB for PostgreSQL as the transactional system of record
- Redesigning the candidate app or employer portal front end
- Changing the scoring rubric or evaluation methodology itself
- Building the natural-language query layer beyond a proof of concept (Section 04)



Ontology & Entity Model

The 14 node types and their full property schema — the vocabulary the rest of this deck builds on.

Ontology Overview: Node Categories

Every node in the VEEP knowledge graph falls into one of five categories, each with a distinct role in the evidence chain.

Actors	`Candidate, Employer, AIPersona`	*Who or what acts within the system*
Structural	`SimulatedCompany, Role, Region`	*The fixed context actors operate inside*
Work Objects	`Task, Artefact`	*What is created as work happens*
Evidence & Trust	`Score, Competency, VerifiedProfile, TrustScore, VerificationHash`	*What proves work happened and how well*
Commercial	`Placement`	*What connects evidence to a business outcome*

Full Node Taxonomy: 14 Node Types

1. Candidate — Actors

A person building a verified profile through the simulation.

2. Employer — Actors

An organisation discovering and hiring verified candidates.

3. AIPersona — Actors

An AI manager, colleague, or evaluator persona driving simulation interactions.

4. SimulatedCompany — Structural

The purpose-built simulated company environment a candidate is placed inside.

5. Role — Structural

One of the 15 role tracks a candidate can select, such as Cybersecurity Analyst.

6. Region — Structural

One of VEEP's three launch markets, calibrating cultural and workplace norms.

7. Task — Work Objects

A unit of work assigned to a candidate by an AI manager persona, tracked to a deadline.

8. Artefact — Work Objects

The candidate's stored deliverable — report, code, dashboard, or design file.

9. Score — Evidence & Trust

The per-competency output of the dual-layer evaluator: rubric plus AI narrative.

10. Competency — Evidence & Trust

A named professional dimension being tested, such as Communication or Risk Prioritisation.

11. VerifiedProfile — Evidence & Trust

The compounding, tamper-proof record an employer ultimately reviews and trusts.

12. TrustScore — Evidence & Trust

The overall professional readiness score, computed by traversal over all evidence.

13. VerificationHash — Evidence & Trust

The tamper-evidence fingerprint attached to every submitted artefact.

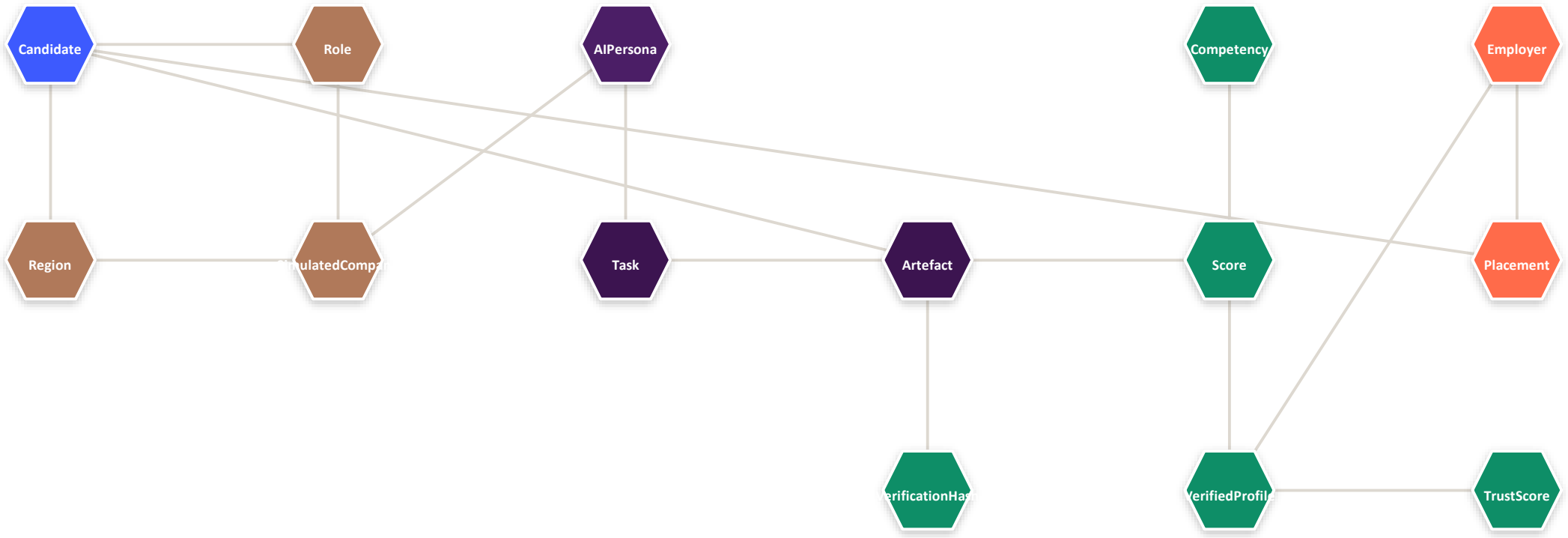
14. Placement — Commercial

A hire transaction and its associated settlement, connecting an employer to a candidate.

Full Relationship Taxonomy: 20 Edge Types

EDGE	DIRECTION	CATEGORY LINK
SELECTED	Candidate → Role	Actor / Structural
PLACED_IN	Candidate → SimulatedCompany	Actor / Structural
STAFFED_BY	SimulatedCompany → AIPersona	Structural / Actor
ASSIGNED	AIPersona → Task	Actor / Work
PRODUCED	Candidate → Artefact	Actor / Work
GENERATED_FROM	Artefact → Task	Work / Work
EVALUATED_BY	Artefact → Score	Work / Evidence
SCORED_BY	Score → AIPersona	Evidence / Actor
MEASURED_BY	Competency → Score	Structural / Evidence
CONTRIBUTES_TO	Score → TrustScore	Evidence / Evidence
VERIFIES	VerificationHash → Artefact	Evidence / Work
AGGREGATES	VerifiedProfile → Score	Evidence / Evidence
SHOWN_ON	TrustScore → VerifiedProfile	Evidence / Evidence
DISCOVERED	Employer → VerifiedProfile	Actor / Evidence
INITIATED	Employer → Placement	Actor / Commercial
CONCERNS	Placement → Candidate	Commercial / Actor
CALIBRATES	Region → SimulatedCompany	Structural / Structural
SCOPES	Region → Candidate	Structural / Actor
TESTS	Role → Competency	Structural / Structural
OPERATES_IN	Employer → Region	Actor / Structural

Master Schema Diagram



Full property detail for every node above is set out individually across the following 14 slides.

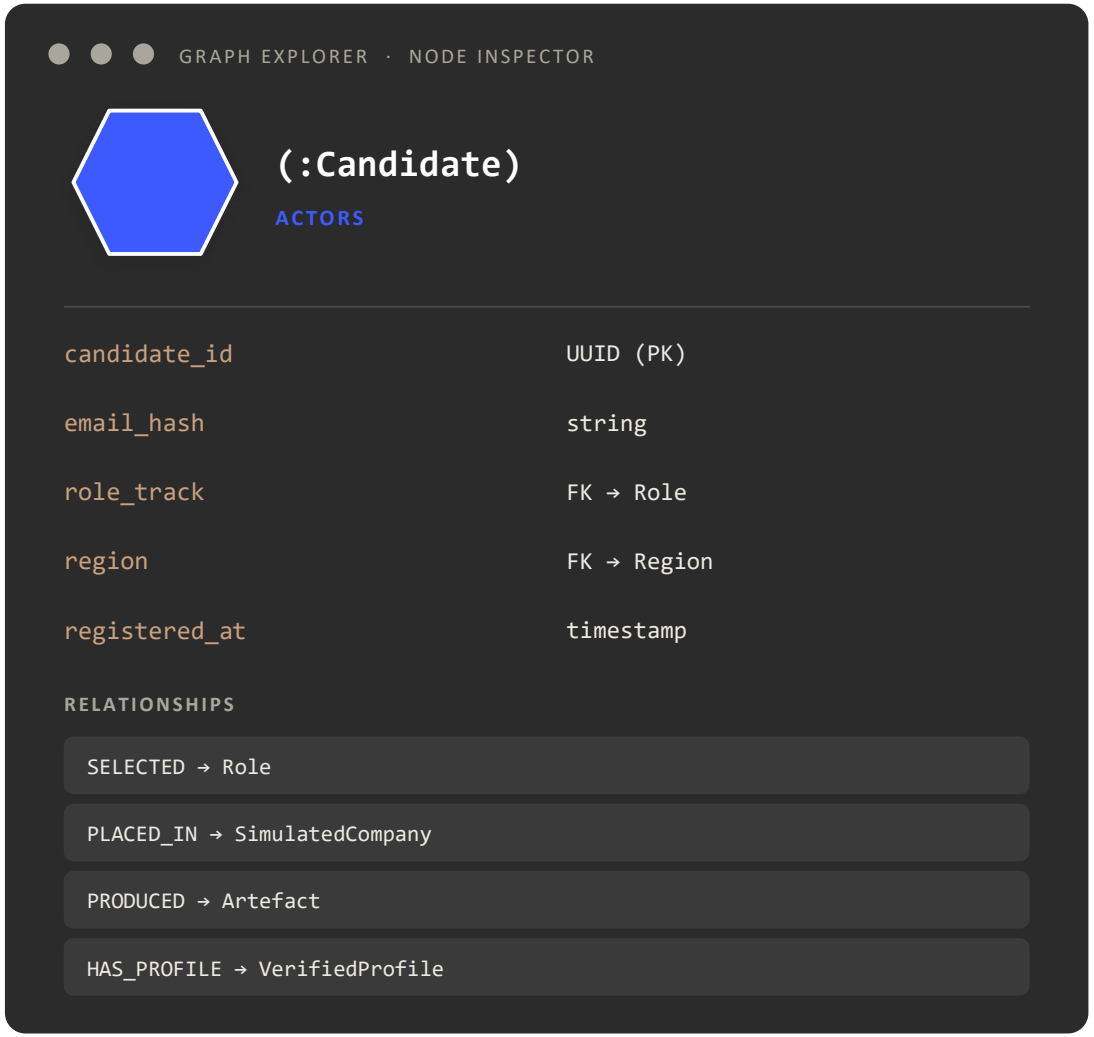
Node: Candidate

A person building a verified profile through the simulation.

CATEGORY: ACTORS

FULL PROPERTY SCHEMA

candidate_id	UUID (PK)
email_hash	string
role_track	FK → Role
region	FK → Region
registered_at	timestamp
status	enum



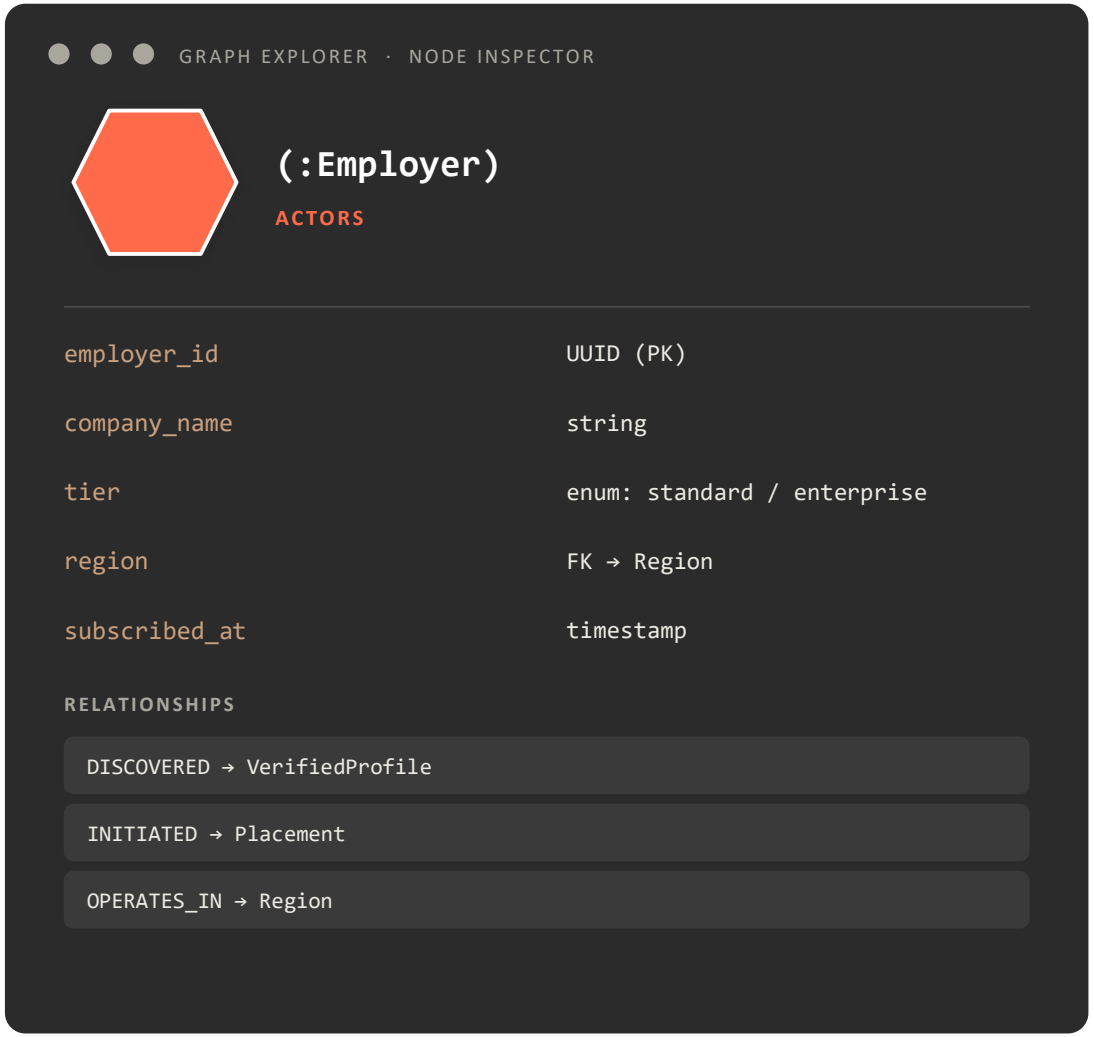
Node: Employer

An organisation discovering and hiring verified candidates.

CATEGORY: ACTORS

FULL PROPERTY SCHEMA

employer_id	UUID (PK)
company_name	string
tier	enum: standard / enterprise
region	FK → Region
subscribed_at	timestamp



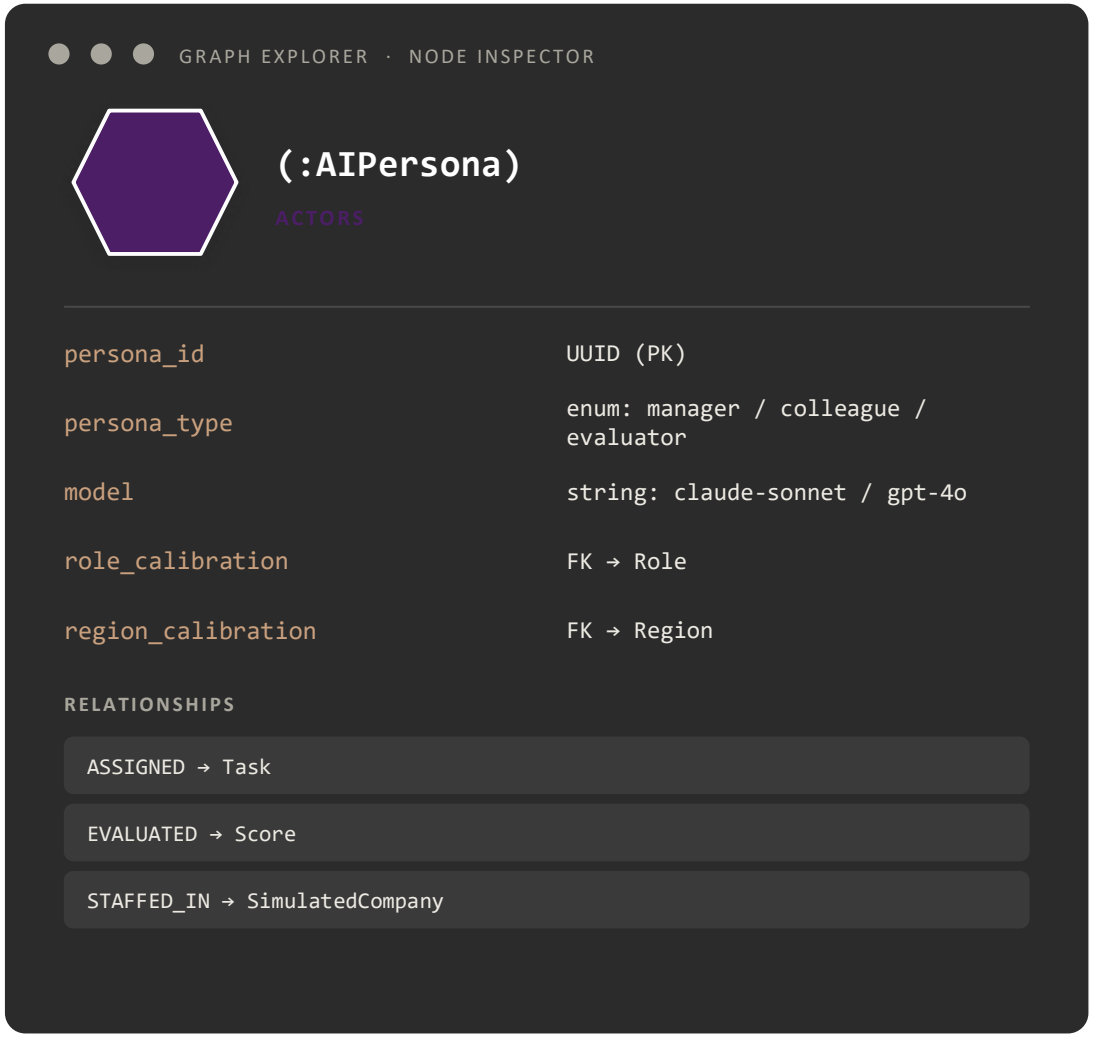
Node: AIPersona

An AI manager, colleague, or evaluator persona driving simulation interactions.

CATEGORY: ACTORS

FULL PROPERTY SCHEMA

persona_id	UUID (PK)
persona_type	enum: manager / colleague / evaluator
model	string: claude-sonnet / gpt-4o
role_calibration	FK → Role
region_calibration	FK → Region



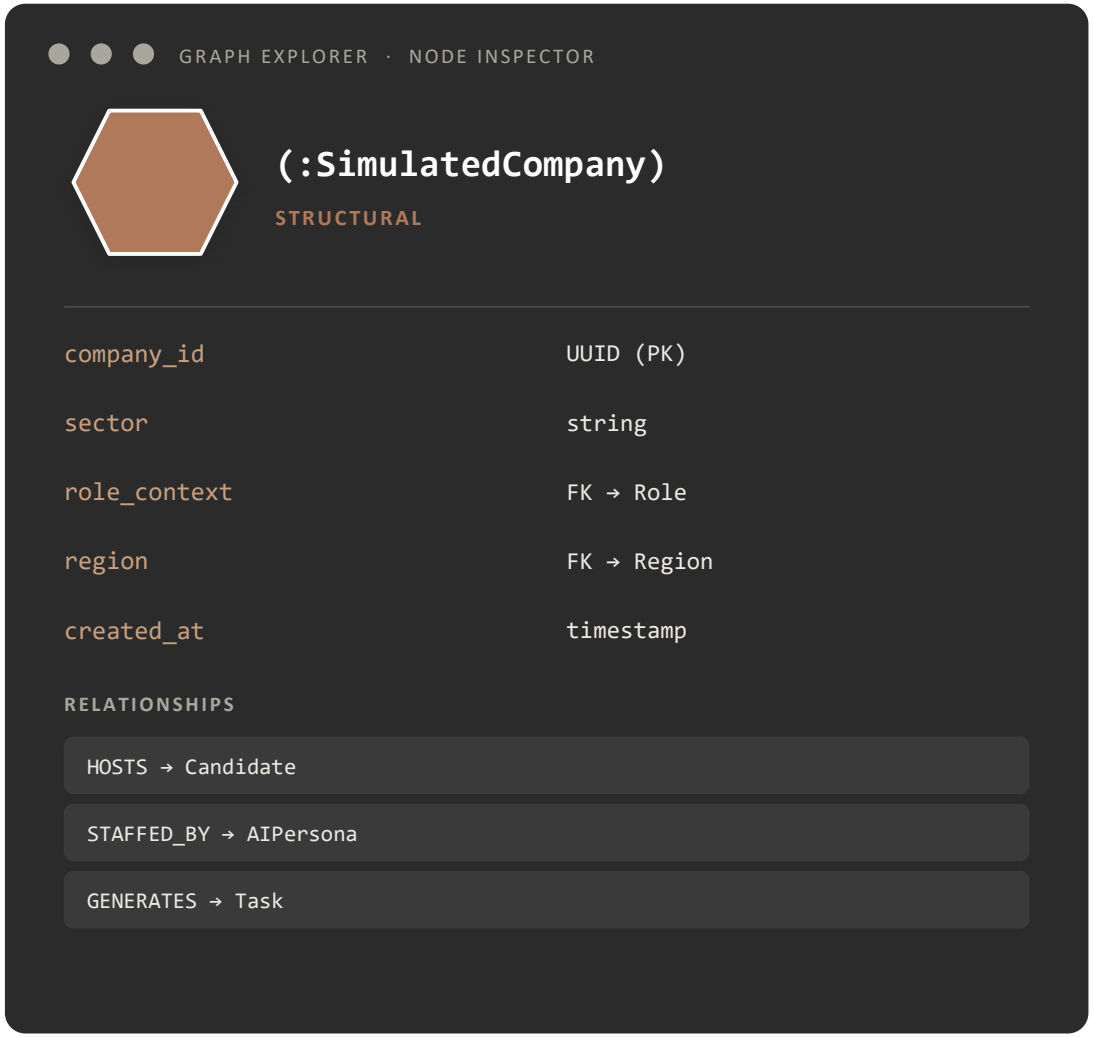
Node: SimulatedCompany

The purpose-built simulated company environment a candidate is placed inside.

CATEGORY: STRUCTURAL

FULL PROPERTY SCHEMA

company_id	UUID (PK)
sector	string
role_context	FK → Role
region	FK → Region
created_at	timestamp



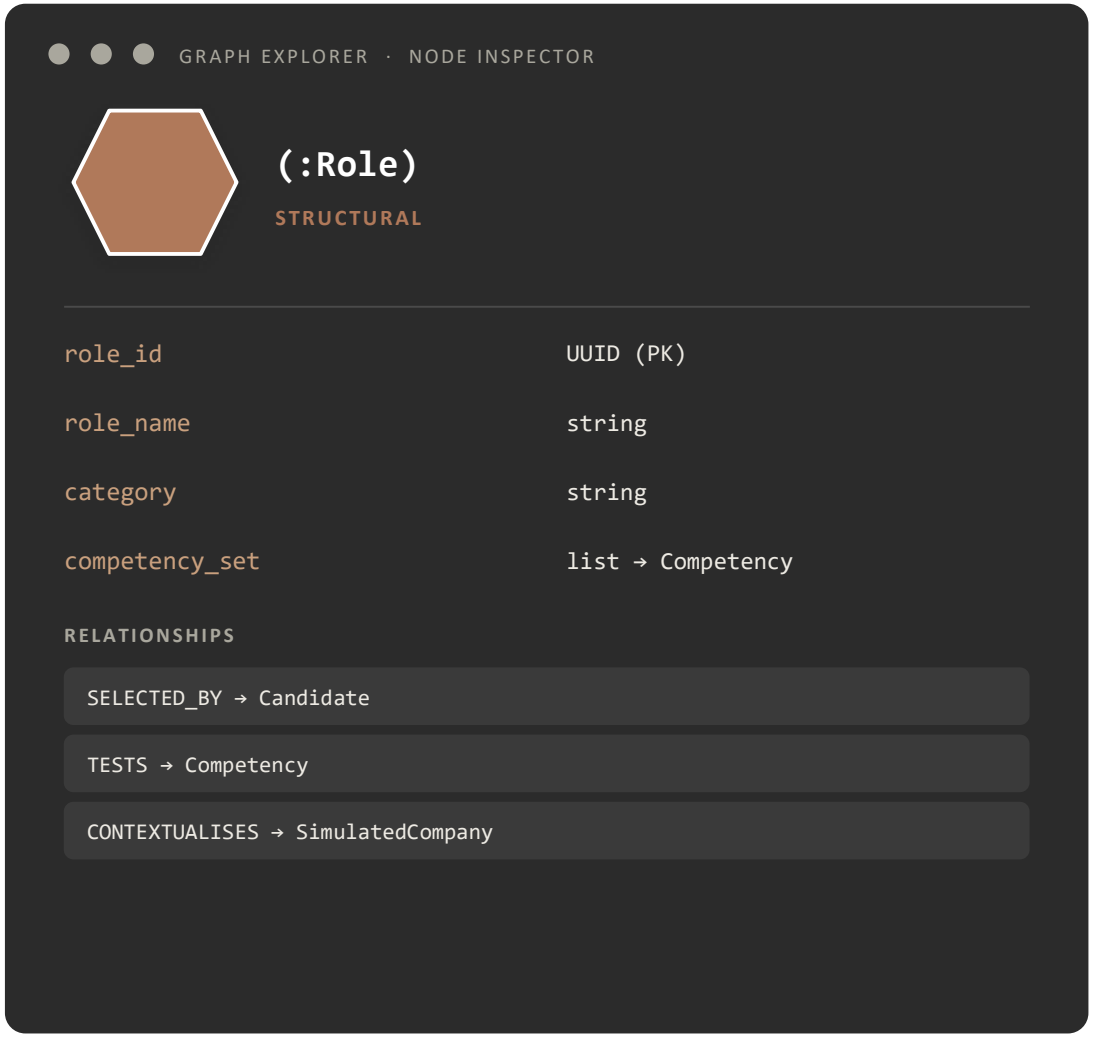
Node: Role

One of the 15 role tracks a candidate can select, such as Cybersecurity Analyst.

CATEGORY: STRUCTURAL

FULL PROPERTY SCHEMA

role_id	UUID (PK)
role_name	string
category	string
competency_set	list → Competency



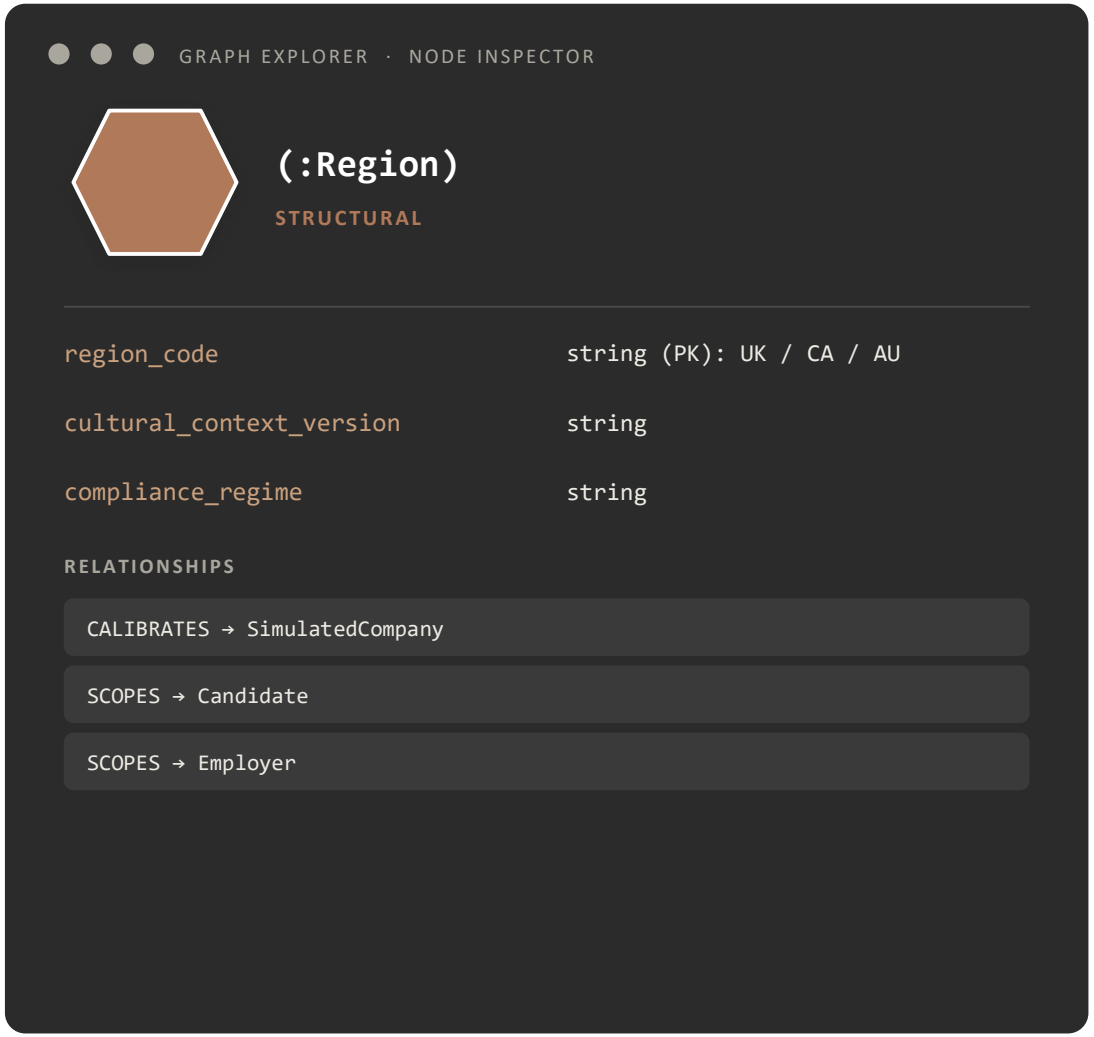
Node: Region

One of VEEP's three launch markets, calibrating cultural and workplace norms.

CATEGORY: STRUCTURAL

FULL PROPERTY SCHEMA

region_code	string (PK): UK / CA / AU
cultural_context_version	string
compliance_regime	string



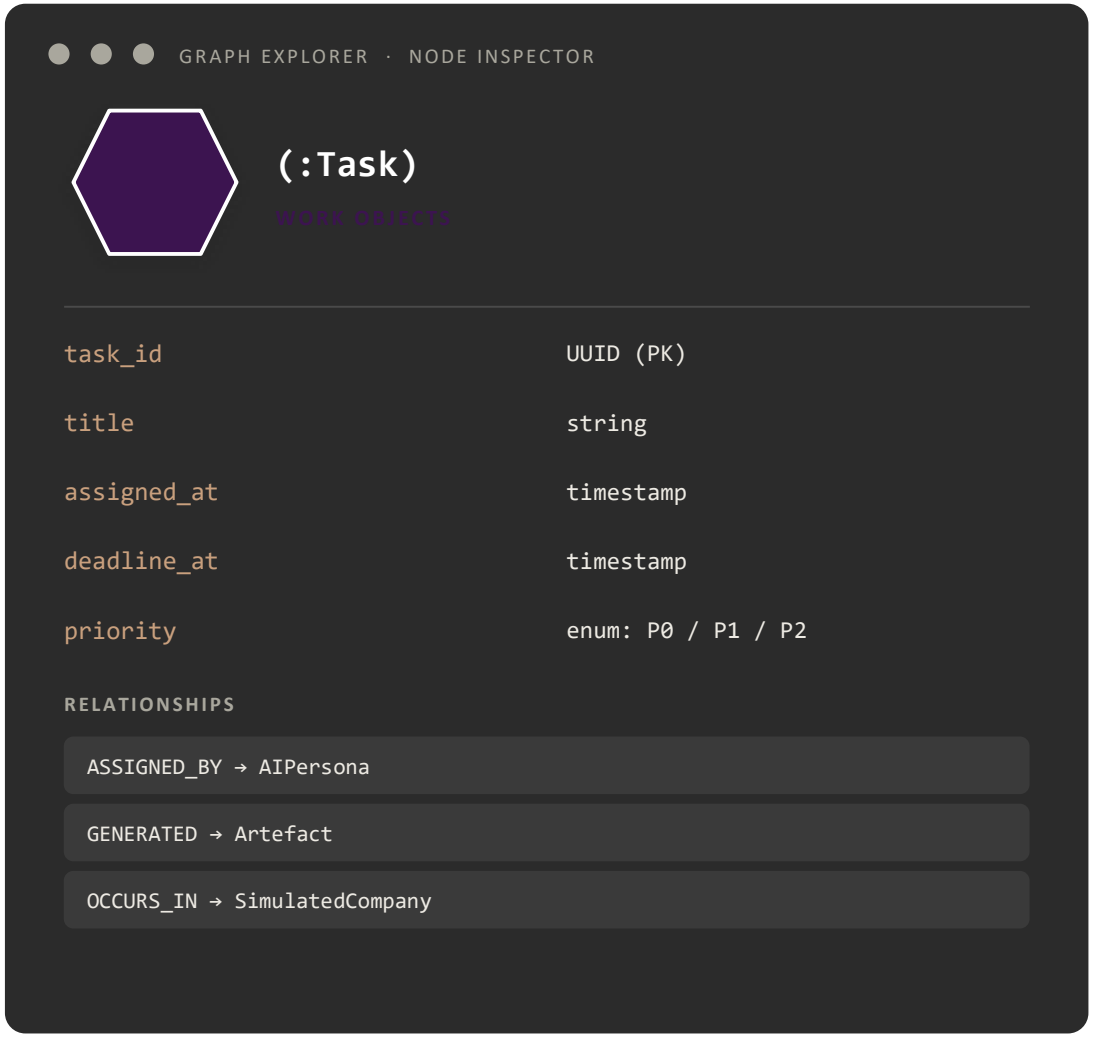
Node: Task

A unit of work assigned to a candidate by an AI manager persona, tracked to a deadline.

CATEGORY: WORK OBJECTS

FULL PROPERTY SCHEMA

task_id	UUID (PK)
title	string
assigned_at	timestamp
deadline_at	timestamp
priority	enum: P0 / P1 / P2



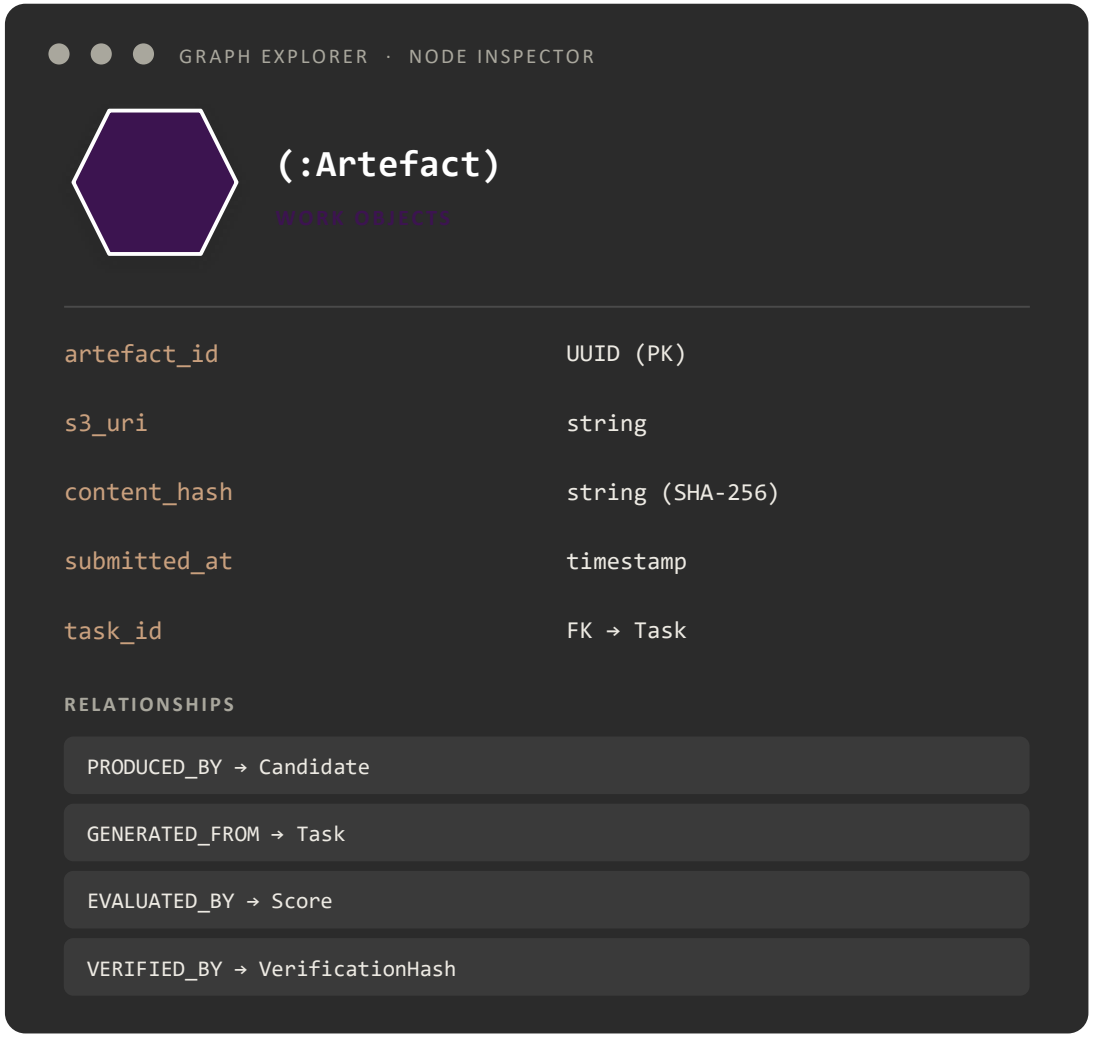
Node: Artefact

The candidate's stored deliverable — report, code, dashboard, or design file.

CATEGORY: WORK OBJECTS

FULL PROPERTY SCHEMA

artefact_id	UUID (PK)
s3_uri	string
content_hash	string (SHA-256)
submitted_at	timestamp
task_id	FK → Task



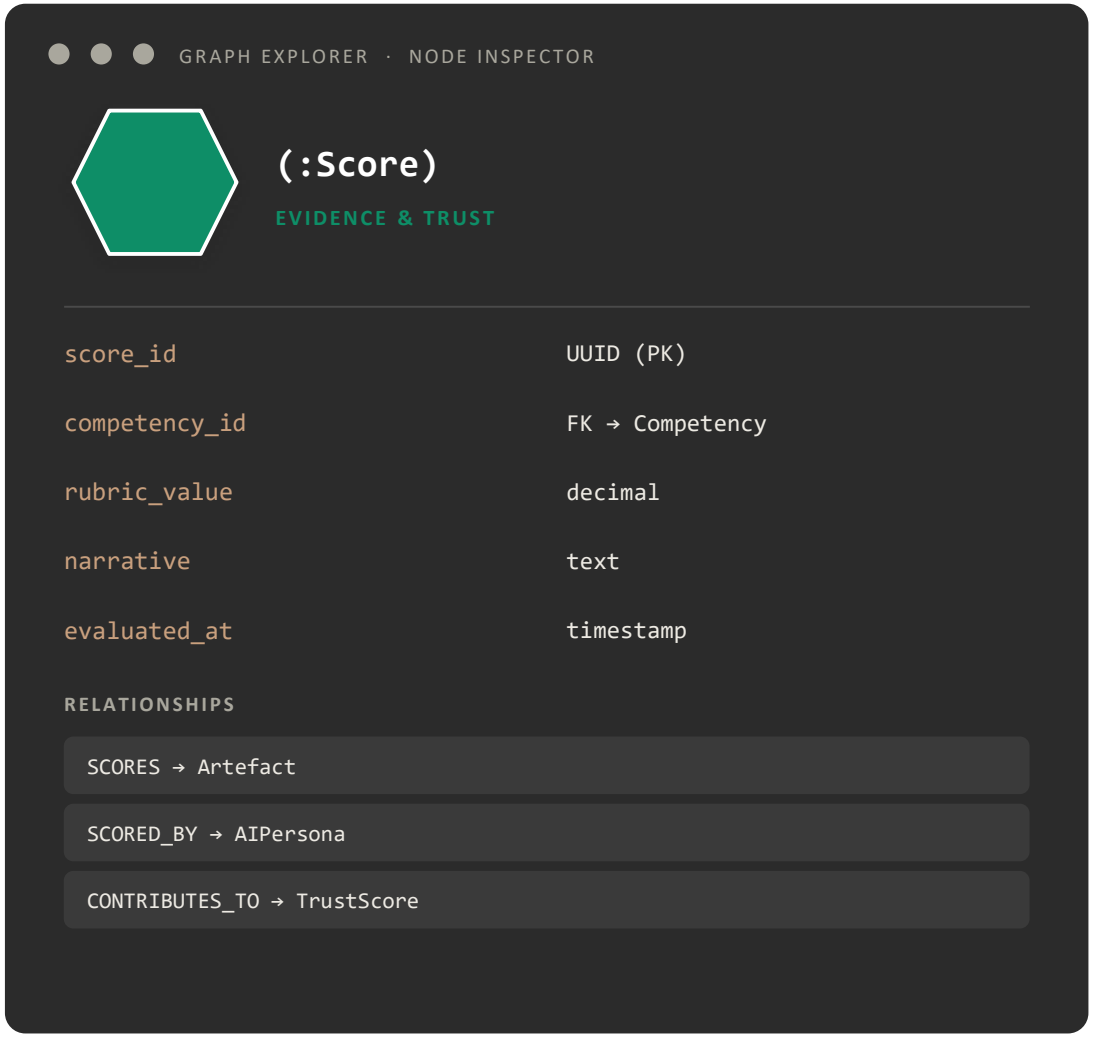
Node: Score

The per-competency output of the dual-layer evaluator: rubric plus AI narrative.

CATEGORY: EVIDENCE & TRUST

FULL PROPERTY SCHEMA

score_id	UUID (PK)
competency_id	FK → Competency
rubric_value	decimal
narrative	text
evaluated_at	timestamp



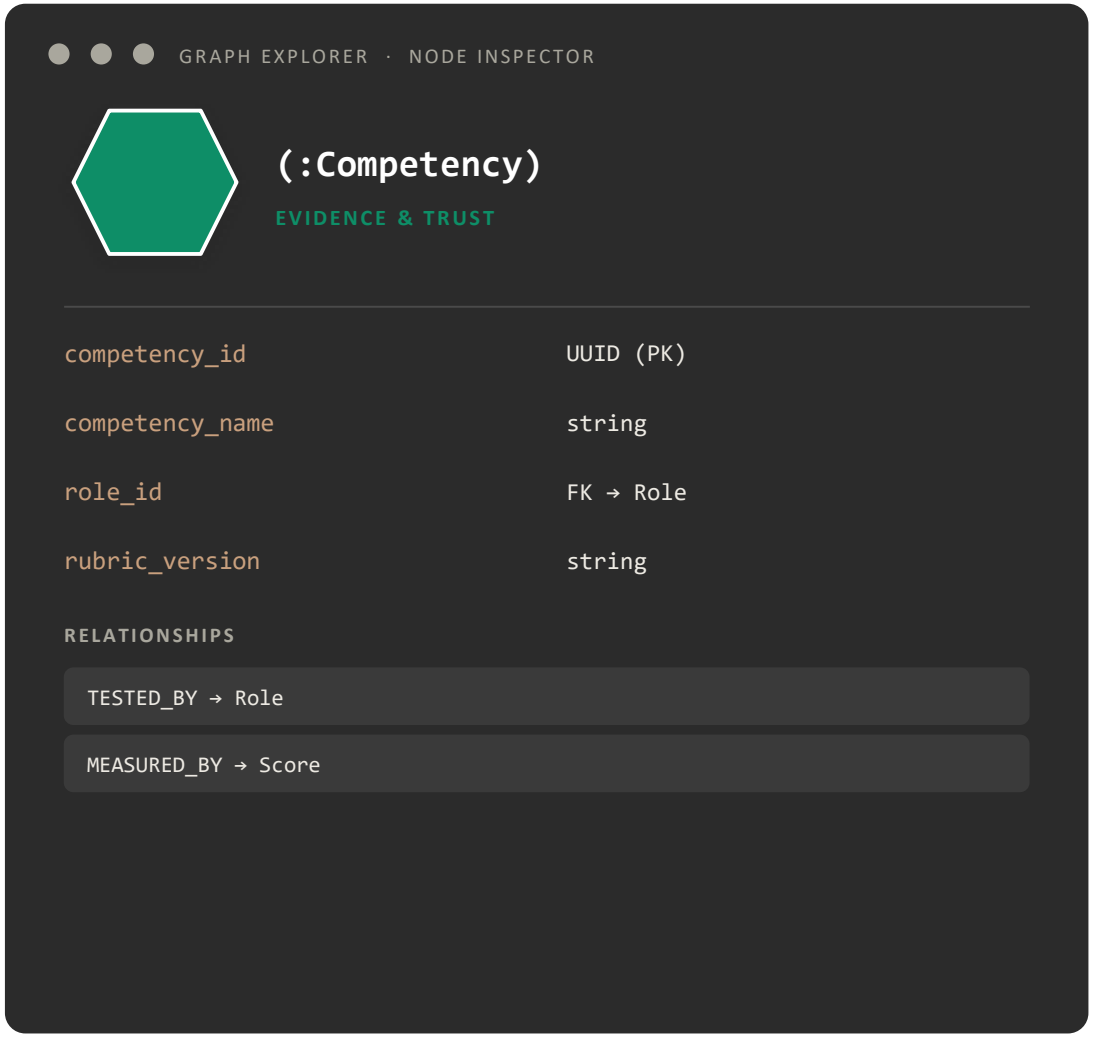
Node: Competency

A named professional dimension being tested, such as Communication or Risk Prioritisation.

CATEGORY: EVIDENCE & TRUST

FULL PROPERTY SCHEMA

competency_id	UUID (PK)
competency_name	string
role_id	FK → Role
rubric_version	string



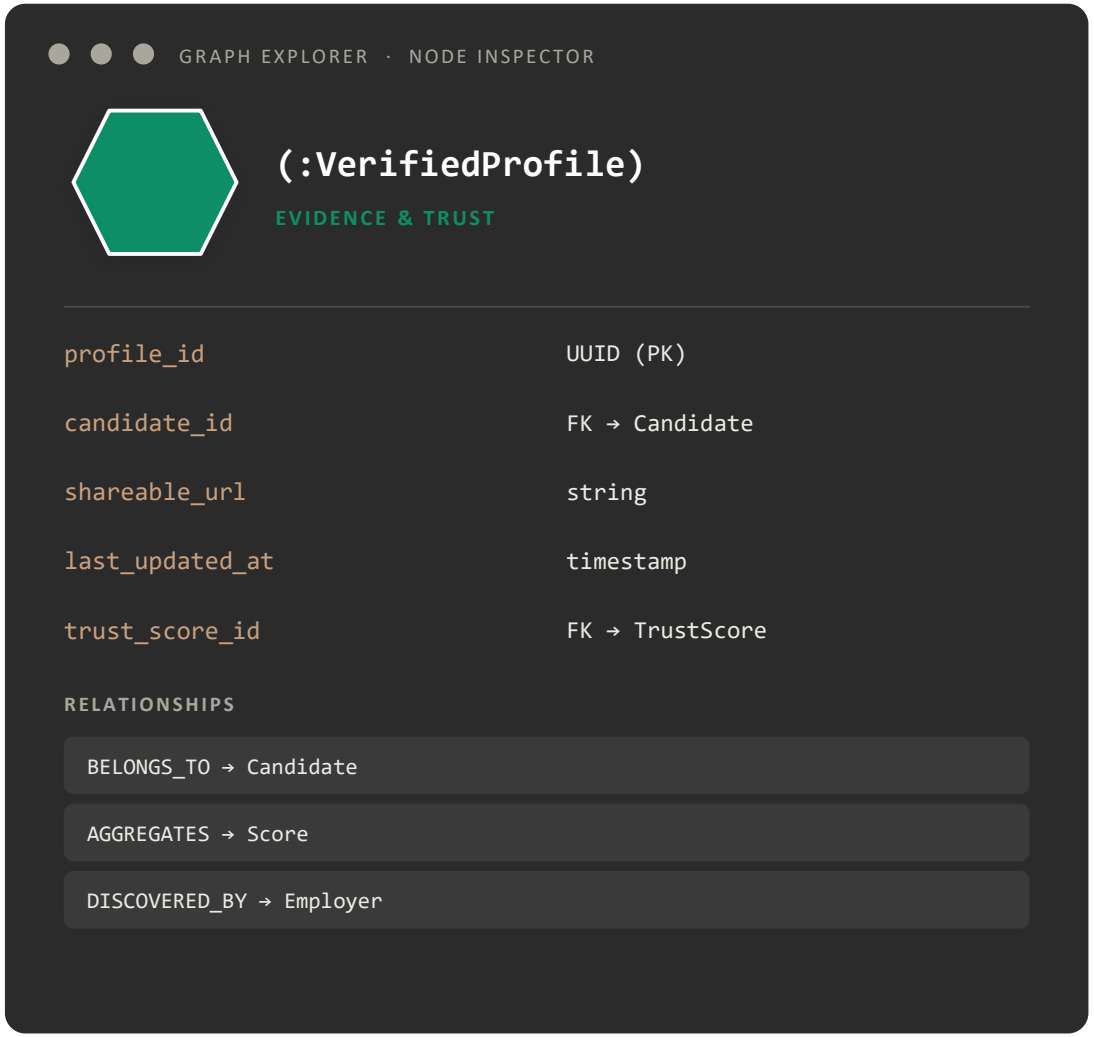
Node: VerifiedProfile

The compounding, tamper-proof record an employer ultimately reviews and trusts.

CATEGORY: EVIDENCE & TRUST

FULL PROPERTY SCHEMA

profile_id	UUID (PK)
candidate_id	FK → Candidate
shareable_url	string
last_updated_at	timestamp
trust_score_id	FK → TrustScore



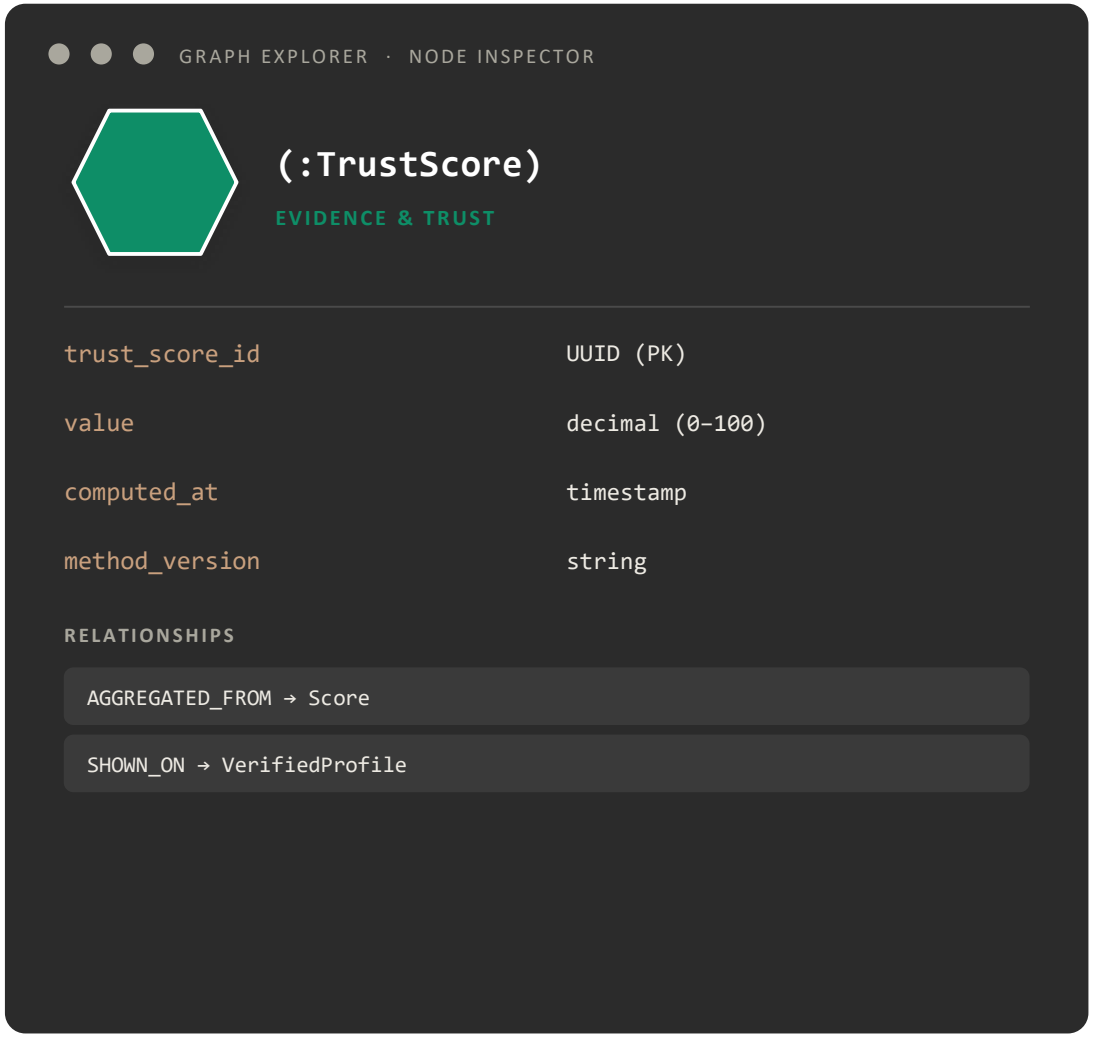
Node: TrustScore

The overall professional readiness score, computed by traversal over all evidence.

CATEGORY: EVIDENCE & TRUST

FULL PROPERTY SCHEMA

trust_score_id	UUID (PK)
value	decimal (0–100)
computed_at	timestamp
method_version	string



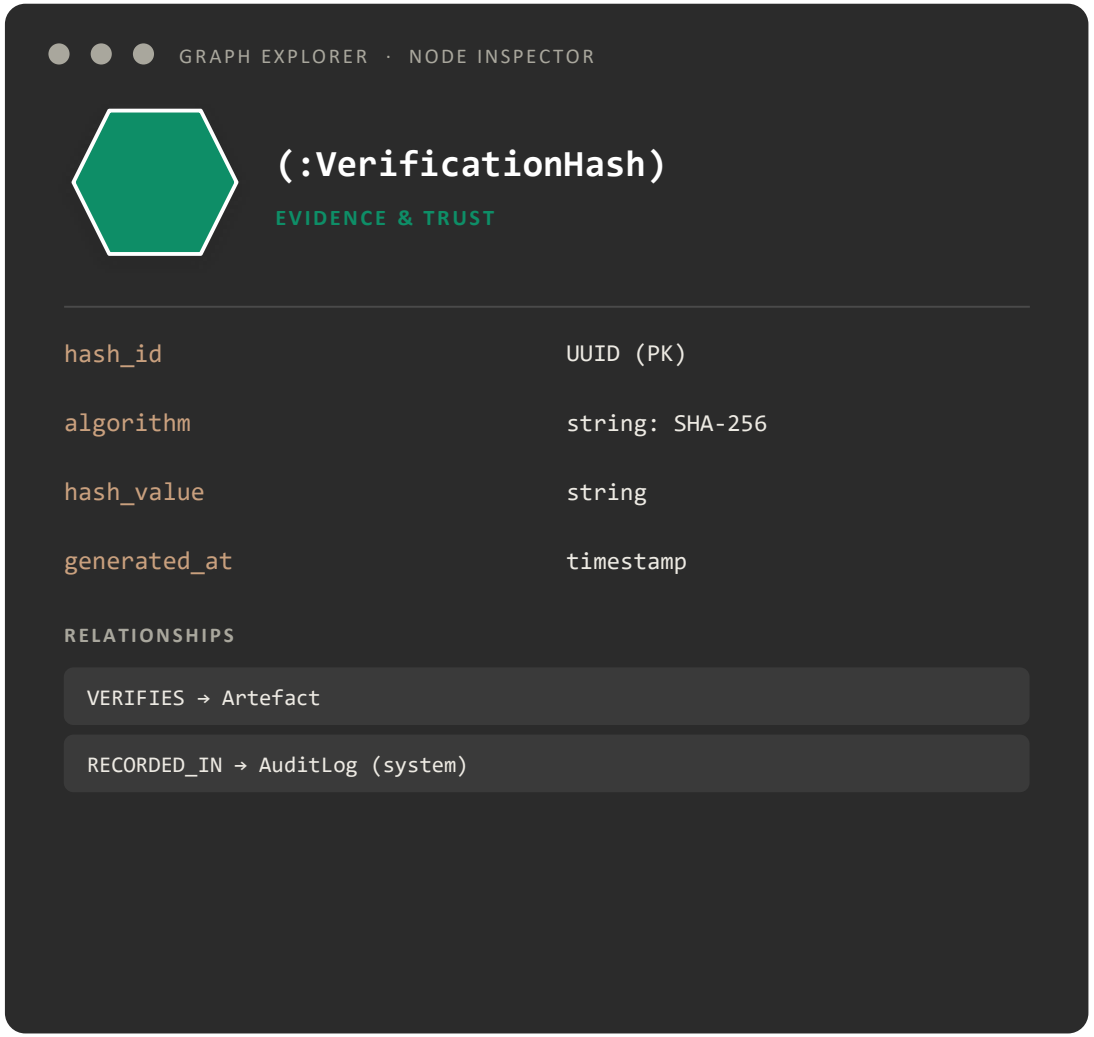
Node: VerificationHash

The tamper-evidence fingerprint attached to every submitted artefact.

CATEGORY: EVIDENCE & TRUST

FULL PROPERTY SCHEMA

hash_id	UUID (PK)
algorithm	string: SHA-256
hash_value	string
generated_at	timestamp



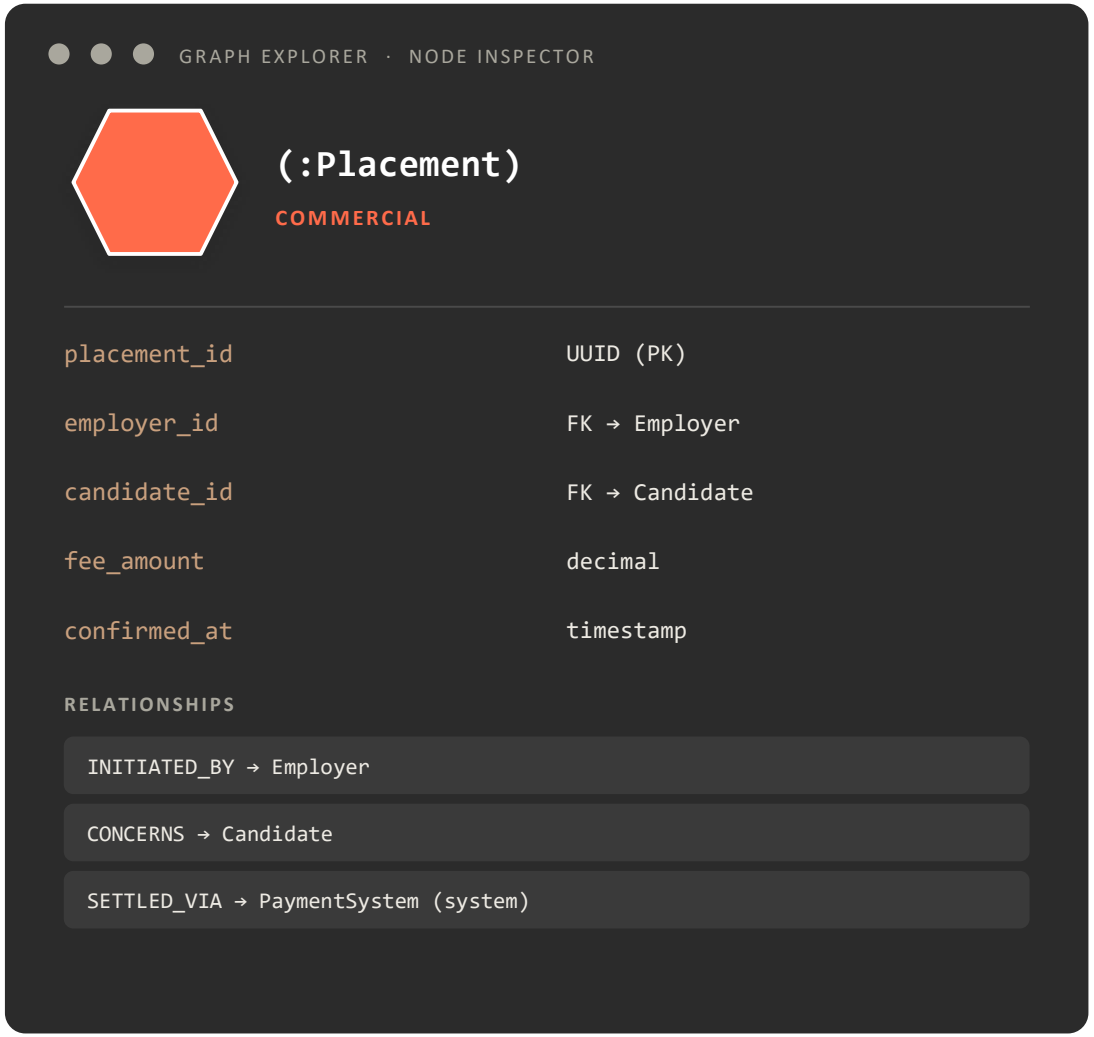
Node: Placement

A hire transaction and its associated settlement, connecting an employer to a candidate.

CATEGORY: COMMERCIAL

FULL PROPERTY SCHEMA

placement_id	UUID (PK)
employer_id	FK → Employer
candidate_id	FK → Candidate
fee_amount	decimal
confirmed_at	timestamp

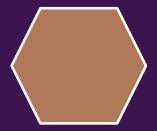


Cross-Node Property Standards

Every node and edge in the graph, regardless of type, carries the same five structural properties, so provenance and lineage queries work identically across the whole ontology.

_id	UUID v4	Globally unique identifier, generated at creation, never reused
_created_at	ISO 8601 timestamp	The moment the node or edge was written to the graph
_source_system	string	Which upstream system wrote this record — e.g. task-engine, evaluator-service
_source_event_id	UUID	The originating event ID in the CDC stream, for full traceability back to AlloyDB
_region	enum: UK / CA / AU	The regional partition this record belongs to, enforced at write time

This is what makes the design principle from Section 01 operational: _source_event_id means no node exists in the graph without a traceable cause.

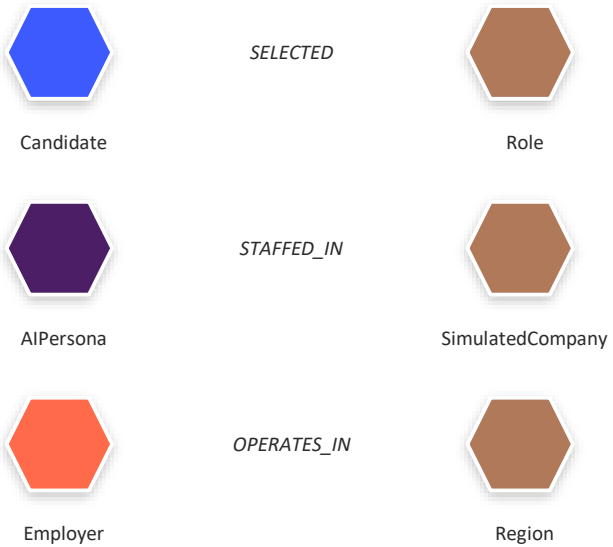


Relationships & Graph Patterns

The full edge model, organised by function, and the traversal patterns VEEP's product runs on.

Relationship Model: Actor Relationships

Relationships that connect the three actor types — Candidate, Employer, AIPersona — to each other and to the structure they operate inside.



Recorded once, at onboarding — the candidate's chosen role track.

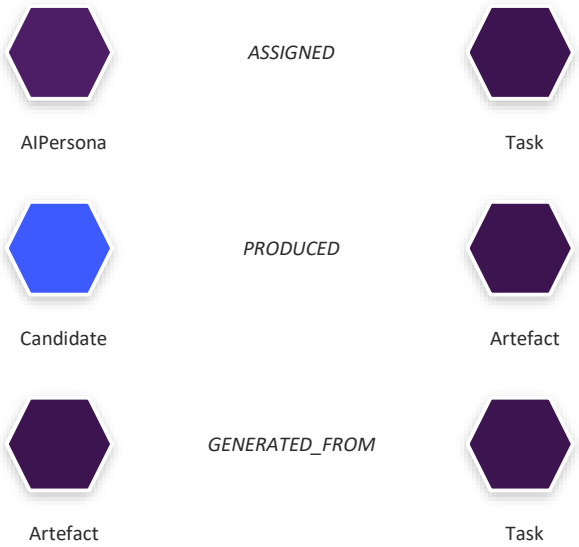
An AI manager or colleague persona is assigned to a specific company instance.

Scopes which candidate pool and compliance regime apply to that employer.

Actor edges typically carry a single timestamp property and are written once — they describe assignment, not ongoing activity.

Relationship Model: Work Relationships

Relationships that connect actors to the work objects — Task and Artefact — they create and act on.



Carries assigned_at and deadline_at; the source event for every downstream task.

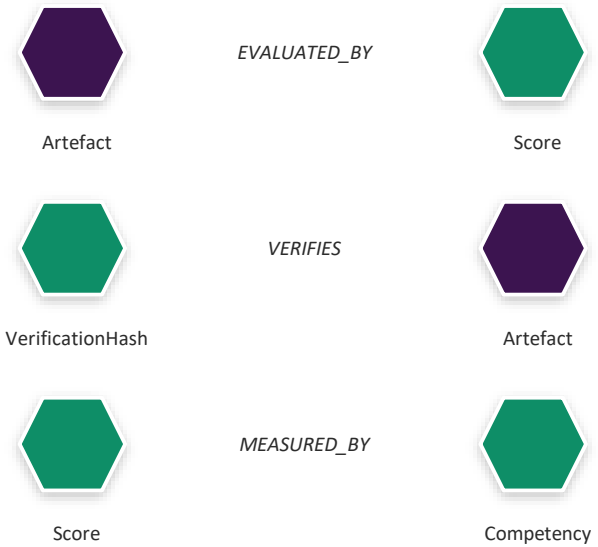
Carries submitted_at; the edge a trust-score traversal must be able to find.

Links a deliverable back to the specific task that requested it.

Work edges are high-volume and time-ordered — they are the edges the CDC ingestion pipeline in Section 05 writes most frequently.

Relationship Model: Evidence Relationships

Relationships that connect work objects to the evidence and verification layer.



The dual-layer evaluator's rubric and narrative output, linked to the exact artefact scored.

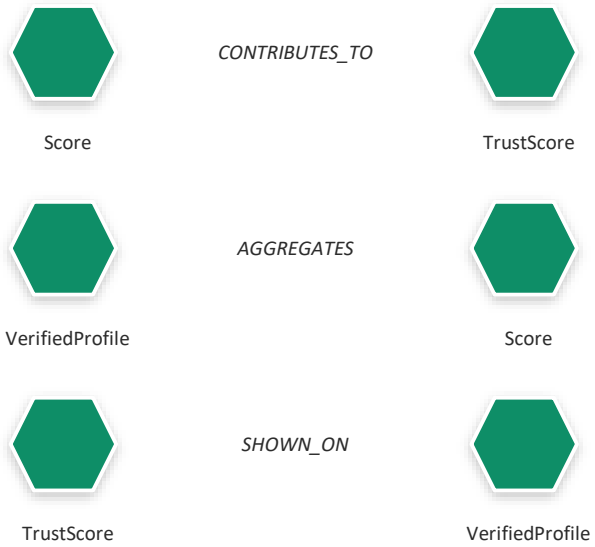
The SHA-256 fingerprint proving the artefact is unaltered since submission.

Ties a numeric score to the named professional dimension it tested.

Every evidence edge is immutable once written — Section 06's governance model treats any edit to an evidence edge as a security incident.

Relationship Model: Trust Relationships

Relationships that aggregate individual scores into the compounding, employer-facing trust picture.



Weighted by competency and recency — the core aggregation edge.

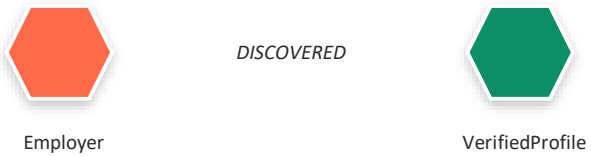
Every score behind a candidate's public profile, in full.

The single computed number an employer sees at the top of the profile.

Trust edges carry a weight property — recomputed on each new score, never manually adjusted, per the guiding principles in Section 01.

Relationship Model: Commercial Relationships

Relationships that connect evidence to a business outcome — discovery, hiring, and settlement.



Logged on every profile view — the raw material for discovery-path analytics.



The moment an employer converts interest into a hire transaction.



Closes the loop from evidence to employment outcome.

Commercial edges are the ones Section 04's recommendation engine reads most, to learn which evidence patterns actually lead to hires.

Relationship Property Schema

Beyond the cross-node standards in Section 02, every edge additionally carries the following properties, used by the query patterns on the following slides.

weight	`decimal (0–1)`	Used on CONTRIBUTES_TO and MEASURED_BY, for weighted aggregation into TrustScore
confidence	`decimal (0–1)`	Used on AI-generated edges (SCORED_BY, EVALUATED_BY) to carry model confidence
valid_from / valid_to	`timestamp`	Supports temporal queries — 'what did this profile look like on this date'
direction	`enum: directed / bidirectional`	Most VEEP edges are directed; a small number (e.g. peer COLLABORATED_WITH) are not

Graph Pattern: Candidate Journey Path

The full candidate journey, from the VEEP Workflow Analysis, expressed as a single traversal from the Candidate node to Placement.



The stages of the workflow that involve AI manager engagement, task execution, and colleague interaction collapse into the Task and Artefact edges above; the graph does not need a separate node per conversational turn to preserve the evidence chain.

Graph Pattern: Employer Discovery Traversal

How an employer's filtered search — competency score, region, trust score, specific skills — resolves to a ranked candidate list.



This traversal runs in reverse from a filter (region, competency) back to a ranked set of candidates — the query pattern the GraphQL discovery API in Section 05 is built around.

Graph Pattern: Evidence Provenance Chain

The exact traversal an auditor or a disputing party uses to prove a piece of evidence is genuine.



This is deliberately the shortest pattern in the deck — provenance queries must be fast and simple enough to run in real time when an employer challenges a piece of evidence.

Graph Pattern: Trust Score Aggregation Path

How TrustScore.value is computed: a weighted traversal across every Score edge contributing to a candidate's profile.



Recomputed on every new Score node — never a stored constant. The weight property (previous slide) determines each score's influence, calibrated per competency and recency.

Graph Pattern: Fraud & Tamper Detection Traversal

A structural anomaly check: does every Artefact behind a high TrustScore have a matching, unaltered VerificationHash?



Run continuously as a background integrity job (Section 07): any TrustScore reachable without a complete, valid hash chain is flagged before it ever reaches an employer.

Find candidates with verified cybersecurity evidence in the UK above trust score 80

OPENCYPHER

```
MATCH (c:Candidate {region: 'UK'})-[:SELECTED]->(r:Role {role_name: 'Cybersecurity Analyst'})
MATCH (c)-[:HAS_PROFILE]->(vp:VerifiedProfile)-[:SHOWN_ON]-(t:TrustScore)
WHERE t.value > 80
MATCH (c)-[:PRODUCED]->(a:Artefact)<-[:VERIFIES]-(h:VerificationHash)
RETURN c, vp, t, collect(a) AS evidence
ORDER BY t.value DESC;
```

WHAT THIS PROVES

This is the query behind the employer discovery filter shown in the Workflow Analysis (competency score, region, trust score). It returns only candidates whose entire evidence set has a valid hash — an unverifiable candidate is structurally excluded, not filtered out after the fact.

Trace every artefact behind a candidate's trust score

OPENCYPHER

```
MATCH (c:Candidate {candidate_id: $id})-[:HAS_PROFILE]->(:VerifiedProfile)
      -[:SHOWN_ON]-(t:TrustScore)
MATCH (t)<-[:CONTRIBUTES_TO]-(sc:Score)<-[:EVALUATED_BY]-(a:Artefact)
MATCH (a)<-[:VERIFIES]-(h:VerificationHash)
RETURN a.artefact_id, a.submitted_at, sc.rubric_value,
       h.hash_value, h.algorithm
ORDER BY a.submitted_at;
```

WHAT THIS PROVES

This is the full disclosure query — every artefact, score, and hash that produced a given trust score, in submission order. It is the query the employer portal calls when an employer clicks 'view evidence' on a candidate's profile.

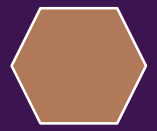
Detect anomalous score patterns across an AI evaluator persona

OPENCYPHER

```
MATCH (p:AIPersona {persona_type: 'evaluator'})-[:EVALUATED]->(sc:Score)
WITH p, avg(sc.rubric_value) AS avgScore, count(sc) AS n
WHERE n > 30 AND avgScore > 92
MATCH (p)-[:EVALUATED]->(sc2:Score)<-[:EVALUATED_BY]-(a:Artefact)
      <-[:PRODUCED]-(c:Candidate)
RETURN p.persona_id, avgScore, n,
       collect(DISTINCT c.candidate_id) AS candidatesInvolved;
```

WHAT THIS PROVES

Flags a single evaluator persona issuing unusually high scores at volume — exactly the multi-hop, aggregation-plus-traversal question raised in Section 01. This runs as a scheduled integrity job feeding the monitoring dashboard in Section 07.



Hybrid Search & AI Integration

How the graph works alongside VEEP's existing pgvector embeddings and Claude/GPT-driven personas.

Why Graph and Vector Search Work Together

Vector search answers 'what is semantically similar to this.' Graph traversal answers 'what is structurally connected to this, and how.' VEEP's matching and evaluation problems need both answered in the same query — neither one alone is sufficient.

VECTOR SEARCH (PGVECTOR)

- Finds candidates whose artefact text reads similarly to a job description
- Ranks by semantic distance, not by any explicit relationship
- Fast at scale, but blind to provenance and trust

GRAPH TRAVERSAL (KNOWLEDGE GRAPH)

- Confirms that a similar-looking candidate has a provable evidence chain
- Filters by exact structural facts: region, verified trust score, role
- Explains why a result was returned, edge by edge

Embedding Integration Architecture

pgvector remains the vector store. The graph stores a reference to each embedding, not the vector itself, keeping the two systems loosely coupled.

Artefact text

AlloyDB for PostgreSQL row

Embedding job

Node.js worker, OpenAI embeddings API

pgvector column

vector(1536) on artefacts table

Artefact node

embedding_ref property (pointer only)

HYBRID QUERY PATTERN — OPENCYPHER + VECTOR PRE-FILTER

```
// Step 1 (pgvector): SELECT artefact_id FROM artefacts
//   ORDER BY embedding <=> $job_desc_embedding LIMIT 200
// Step 2 (graph): MATCH (a:Artefact) WHERE a.artefact_id IN $ids
//   MATCH (a)<-[:PRODUCED]-(c:Candidate)-[:HAS_PROFILE]->(:VerifiedProfile)-[:SHOWN_ON]-(t:TrustScore)
//   WHERE t.value > 75  RETURN c, t ORDER BY t.value DESC
```

Candidate–Employer Similarity Matching Pipeline

1	**Employer posts a role brief**	Free-text description of the ideal candidate
2	**Brief is embedded**	OpenAI embeddings API, stored transiently — not persisted
3	**Vector pre-filter**	pgvector returns top 200 semantically similar artefacts
4	**Graph structural filter**	Knowledge graph filters to verified, in-region, trust-scored candidates
5	**Ranked result set**	Returned to employer portal with full evidence links

AI Persona Context Assembly via Graph Traversal

Before an AI manager or colleague persona generates its next message, it needs the candidate's task history and prior evaluation context. That context is assembled with a graph traversal, not a hand-built prompt, so it is always complete and current.

CONTEXT ASSEMBLY SERVICE — SEQUENCE

1. AIPersona.generateNextMessage(candidate_id, task_id) is invoked
2. Graph traversal: MATCH (c:Candidate)-[:PRODUCED]->(a:Artefact) WHERE a.task_id = $task_id
3. Graph traversal: MATCH (a)-[:EVALUATED_BY]->(sc:Score) — retrieves prior scores for this task
4. Graph traversal: MATCH (c)-[:HAS_PROFILE]->(:VerifiedProfile)-[:SHOWN_ON]-(t:TrustScore) — retrieves running trust
5. Assembled context is passed to the Claude API as structured system context, not free text
6. Persona response is generated, logged, and written back as a new graph edge

Graph-Grounded Evaluation: Reducing Hallucination in Scoring

The dual-layer evaluator's AI narrative component is grounded by requiring every claim it makes to cite a specific, retrievable graph node — a task requirement, a prior score, or a rubric definition — rather than generating from memory alone.

WITHOUT GRAPH GROUNDING

- Evaluator narrative is generated from the artefact text alone
- No structural check that the rubric being applied matches the task
- Risk of scoring against the wrong competency version

WITH GRAPH GROUNDING

- Evaluator retrieves the exact Competency node and rubric_version via MEASURED_BY
- Narrative must reference retrieved node properties, checked post-hoc
- Any narrative claim without a matching graph reference is rejected before scoring

Explainability: Graph Path as Evidence Trail

Because a trust score is a traversal result, not a stored number, the exact path that produced it can always be shown to a candidate or an employer on request.

EXPLAIN THIS SCORE — UI MOCK-UP

Trust Score: 87

Computed 14 Jul 2026, from 6 contributing scores

Incident Response Report

Artefact · submitted 2 Jul

Risk Prioritisation: 91/100

Score · rubric v3.2

Weighted 22%

into TrustScore

Every figure on this panel is a live graph traversal result — not cached copy that can drift from the underlying evidence.

Recommendation Engine Architecture

Recommends roles to candidates and candidates to employers by learning which graph patterns preceded successful Placement nodes historically.

1	**Signal collection**	DISCOVERED, INITIATED, and CONCERNS edges are the training signal
2	**Pattern extraction**	Batch job identifies common sub-graphs preceding a Placement
3	**Scoring function**	New candidate–employer pairs scored against learned patterns, blended with vector similarity
4	**Serving**	Top-ranked recommendations surfaced in employer portal and candidate app

Natural Language to Graph Query Layer

A constrained, LLM-assisted layer that translates an employer's plain-English search into a validated GQL query — scoped to proof-of-concept for launch, expanded post-launch per the roadmap in Section 08.

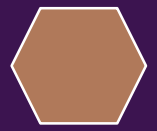
Employer types	*"Verified analysts in Canada, strong in incident response, hired before"*
Claude API (constrained)	Maps phrase to known node/edge/property names only — cannot invent schema
Query validator	Rejects any generated query touching a node or edge outside the published ontology
Spanner Graph	Executes validated query, returns results

Safety property: the LLM never writes to the graph and never executes a query directly — it only proposes, and the validator only allows read-only, schema-conformant queries through.

Data Quality Rules Enforced at Ingestion

Every write to the graph passes through validation before it is committed. These rules exist so hybrid retrieval never returns an incomplete or contradictory result.

Referential completeness	An edge cannot be written if either endpoint node does not yet exist
Hash presence	An Artefact node cannot be marked EVALUATED_BY a Score without a VERIFIES edge from a VerificationHash
Single active TrustScore	A Candidate may have only one current TrustScore node; prior ones are marked historical, never deleted
Region consistency	A Candidate's _region property must match the _region of every SimulatedCompany it is PLACED_IN



Technology Architecture

Graph database selection and the full target-state system design on VEEP's existing GCP footprint.

Graph Database Evaluation Criteria

Six criteria, weighted by relevance to VEEP's operating constraints — a small engineering team, an existing GCP footprint, and a launch timeline measured in months.

GCP-native operation	HIGH	Fits existing VPC, IAM, and Identity Platform setup without a new cloud account
Managed service availability	HIGH	Small team cannot operate self-hosted graph infrastructure
Query language maturity	HIGH	GQL query-language maturity, for the patterns in Section 03
Cost at launch scale	MEDIUM	Must run affordably at hundreds, not millions, of nodes initially
Hybrid vector support	MEDIUM	Native or adjacent vector capability, for Section 04's hybrid queries
Team learning curve	MEDIUM	Existing Data Engineer and AI Engineer must be productive within weeks

PLATFORM COMPARATIVE ANALYSIS

Evaluating Graph Database Platforms Across Cloud Providers

Before finalising Google Cloud Spanner Graph as the platform of record, this section evaluates six graph database options across three hyperscaler ecosystems and three database-native, multi-cloud vendors — scored against the same six criteria on the preceding slide, weighted toward VEEP's three-region residency requirement and small-team operating model.

HYPERSCALER-NATIVE PLATFORMS

- Google Cloud Spanner Graph — Google's globally-distributed, strongly-consistent graph layer built on Spanner
- Amazon Neptune — AWS's managed graph database, supporting openCypher, Gremlin and SPARQL
- Azure Cosmos DB (Gremlin API) — Microsoft's multi-model database with a Gremlin-compatible graph API

DATABASE-NATIVE, MULTI-CLOUD PLATFORMS

- Neo4j AuraDB — the reference implementation of Cypher, offered as a managed service on all three hyperscalers
- ArangoDB (Oasis) — a multi-model database with a native graph engine and proprietary AQL query language
- TigerGraph Cloud — a purpose-built analytical graph platform, strongest at deep multi-hop traversals

METHODOLOGY

Each platform is scored 1-5 against the six criteria on the preceding slide; hyperscaler-native platforms are compared head-to-head, then database-native platforms compared head-to-head, before a combined weighted scorecard.

Comparison Scoring Rubric

Each of the six criteria is scored 1-5 per platform, then multiplied by its weight to produce the weighted totals in this section's scorecard.

GCP / cloud-native fit	×3	5 = zero new account or IAM model to operate; 1 = an entirely separate cloud footprint
Managed service maturity	×3	5 = fully managed with mature SLAs; 1 = significant self-hosted operational burden
Query language & standards	×3	5 = ISO-standard or portable; 1 = a small proprietary ecosystem
Cost at launch scale	×2	5 = predictable and low at hundreds of nodes; 1 = expensive or unpredictable at small scale
Hybrid vector support	×2	5 = native vector search; 1 = no vector capability, external store required
Team learning curve	×2	5 = existing team productive within days; 1 = a specialist hire is required

Query Language & Standards — Hyperscaler-Native Platforms

Criterion	Google Cloud Spanner Graph	Amazon Neptune	Azure Cosmos DB (Gremlin)
Primary language	GQL (ISO/IEC 39075) + SQL	openCypher, Gremlin, SPARQL	Gremlin (Apache TinkerPop)
Standards body	ISO/IEC, ratified 2024	Community specs (openCypher, W3C SPARQL)	Apache TinkerPop
SQL interoperability	Native — graph and relational queries in one statement	None — separate query surface	None — separate query surface
Driver / SDK maturity	Growing — GA since 2024	Mature — GA since 2018	Mature — GA since 2017
Portability from Section 03	Cypher-like syntax, low migration cost	Patterns as originally authored	Requires a Gremlin rewrite
VEEP fit	High	High	Medium

Query Language & Standards — Database-Native Platforms

Criterion	Neo4j AuraDB	ArangoDB (Oasis)	TigerGraph Cloud
Primary language	Cypher	AQL (proprietary)	GSQL (proprietary)
Standards body	Origin of the openCypher/GQL lineage	None — ArangoDB-specific	None — TigerGraph-specific
SQL interoperability	None — separate query surface	None — separate query surface	None — separate query surface
Driver / SDK maturity	Mature — largest graph ecosystem	Moderate — smaller community	Moderate — analytics-focused community
Portability from Section 03	Closest match to patterns as authored	Full rewrite required	Full rewrite required
VEEP fit	High	Low	Low

Managed Service Maturity & Operations — Hyperscaler-Native

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Service model	Fully managed, serverless scaling	Fully managed, provisioned capacity	Fully managed, serverless or provisioned
Patch & upgrade cadence	Continuous, zero-downtime	Scheduled maintenance windows	Continuous, zero-downtime
Overhead for a 3-person team	Minimal — no capacity planning at launch scale	Low — some instance-sizing decisions	Minimal — consumption-based
Support SLA	99.999% (multi-region)	99.9% (Multi-AZ)	99.999% (multi-region)
Production maturity	GA 2024, newest of the three	GA 2018, most established	GA 2017, most established
VEEP fit	High	High	Medium

Managed Service Maturity & Operations — Database-Native

Criterion	Neo4j AuraDB	ArangoDB Oasis	TigerGraph Cloud
Service model	Fully managed, fixed instance tiers	Managed tier via Oasis, self-hosted also available	Fully managed, credit-based
Patch & upgrade cadence	Scheduled, vendor-controlled	Scheduled, vendor-controlled	Scheduled, vendor-controlled
Overhead for a 3-person team	Low	Medium — smaller managed-tier footprint	Medium — less familiar operating model
Support SLA	99.95% (Enterprise tier)	99.9% (Oasis)	99.9%
Production maturity	GA 2016, widely adopted	GA 2018, moderate adoption	GA 2019, niche adoption
VEEP fit	Medium	Low	Low

Consistency & Replication Model — Hyperscaler-Native

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Replication	Synchronous, Paxos consensus across zones/regions	Asynchronous, writer + up to 15 read replicas	Tunable, five consistency levels
Failover behaviour	No promotion step — remaining replicas serve immediately	Reader promoted to writer, under 30 seconds	Automatic, region-aware
RPO on failure	Zero	Near-zero, bounded by replication lag	Zero to seconds, by consistency level chosen
Cross-region consistency	Strong, externally consistent	Eventually consistent across regions	Configurable per request
Operational complexity	Low — no consistency tuning required	Medium — replica topology decisions	Medium — consistency level chosen per workload
VEEP fit	High — matches trust-score integrity requirement	Medium	Medium

Consistency & Replication Model — Database-Native

Criterion	Neo4j AuraDB	ArangoDB	TigerGraph Cloud
Replication	Causal clustering, leader + followers	Active failover, leader + followers	Replicated cluster, configurable factor
Failover behaviour	Automatic leader election	Automatic, brief unavailability window	Automatic, brief unavailability window
RPO on failure	Near-zero	Near-zero	Near-zero
Cross-region consistency	Causal consistency	Eventual, unless configured otherwise	Eventual
Operational complexity	Medium	Medium	Medium-high
VEEP fit	Medium	Low	Low

Native Vector & Hybrid Search Support — Hyperscaler-Native

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Native vector index	Yes — integrated with Spanner vector search	Yes — via Neptune Analytics	Yes — Cosmos DB vector search
Combined graph + vector query	Single GQL/SQL statement	Separate Neptune Analytics call	Separate query surface
Embedding storage	Co-located with graph data	Requires a Neptune Analytics graph copy	Co-located with document/graph data
Fit with Section 04 hybrid design	Direct fit — no extra hop	Adds an analytics-tier hop	Adds a query-surface hop
Maturity	New capability, GA 2024	Newer capability, GA 2023	Newest, still stabilising
VEEP fit	High	Medium	Medium

Native Vector & Hybrid Search Support — Database-Native

Criterion	Neo4j AuraDB	ArangoDB	TigerGraph Cloud
Native vector index	Yes — native vector index since 5.x	Yes — native vector index	No native index — external store required
Combined graph + vector query	Single Cypher statement	Single AQL statement	Not supported natively
Embedding storage	Co-located with graph data	Co-located with graph data	External, adds integration work
Fit with Section 04 hybrid design	Direct fit	Direct fit	Poor fit — extra pipeline required
Maturity	Mature, GA 2023	Mature, GA 2023	Not available
VEEP fit	Medium	Low	Low

Identity & Access Integration — Hyperscaler-Native

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Native IAM	Cloud IAM — same model as AlloyDB and Cloud Storage	AWS IAM — same model as Aurora and S3	Microsoft Entra ID
Service account model	Per-service accounts, least privilege	Per-service IAM roles, least privilege	Managed identities, least privilege
Credential-free auth	Yes — IAM database authentication	Yes — IAM database authentication	Yes — Entra-integrated
Fit with VEEP's auth design (Sec. 05)	Direct — Identity Platform replaces Cognito, same pattern	Direct — original design baseline	Requires a new identity provider integration
VEEP fit	High	High	Low
Overall	Recommended	Prior baseline	Not recommended

SLA & Multi-Region Availability Guarantees

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Published SLA	99.999% (multi-region config)	99.9% (Multi-AZ)	99.999% (multi-region config)
Multi-region write support	Native, synchronous	Not native — needs app-level replication	Native, multi-master
Zero-downtime maintenance	Yes	Partial — brief failover windows	Yes
Read scaling	Automatic across regions	Manual reader-replica provisioning	Automatic across regions
Failover design burden	None — handled transparently	Requires the HA design in Section 05	None — handled transparently
VEEP fit	High — matches UK/Canada/Australia goal	Medium	High

Data Residency & Regional Segregation Fit

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Regional data pinning	Regional or multi-regional instance config	Per-region cluster, as designed in Section 05	Per-region account or regionally pinned writes
UK/Canada/Australia coverage	All three supported (europe-west2, northamerica-northeast1, australia-southeast1)	All three supported, as designed	All three supported
Cross-region query isolation	Enforced at instance boundary	Enforced at cluster boundary	Enforced at account/region boundary
Compliance tooling	Access Transparency, Assured Workloads	AWS Artifact, Config	Compliance Manager
VEEP fit	High	High	Medium
Overall	Recommended	Prior baseline	Not recommended

CDC & Ingestion Pipeline Compatibility

Criterion	Spanner Graph (GCP)	Neptune (AWS)	Cosmos DB (Azure)
Source-database CDC service	Datastream, reading AlloyDB for PostgreSQL logs	Database Migration Service, reading Aurora logs	Azure Data Factory / Debezium
Event streaming layer	Pub/Sub	Kinesis Data Streams	Event Hubs
Compute for ingestion workers	Cloud Run functions	Lambda	Azure Functions
Fit with VEEP's source DB (Sec. 05/06)	Direct — same pipeline shape as this deck's build plan	Original design baseline	Requires migrating the source database too
Rework required	Terminology only — same pipeline shape	None — original design	Full pipeline redesign
VEEP fit	High	High	Low

Pricing Model Structure — Hyperscaler-Native

Criterion	Spanner Graph	Neptune	Cosmos DB (Gremlin)
Compute unit	Processing units (PUs); 100 PU = 1/10 node	Provisioned instance (NCU-hour)	Request Units (RU/s)
Minimum viable footprint	100 PUs — fractional node, low launch cost	Smallest provisioned instance (db.r6g.large)	400 RU/s minimum per container
Scaling model	Add PUs incrementally, no re-provisioning	Resize or add read replicas	Autoscale RU/s, billed on peak
Storage cost	Per GB, replication included	Per GB-month, replication included	Per GB, replication included
Predictability at hundreds of nodes	High — smallest unit is fractional	Medium — smallest instance is a fixed size	Medium — RU/s varies with query shape
VEEP fit	High	Medium	Medium

Pricing Model Structure — Database-Native

Criterion	Neo4j AuraDB	ArangoDB Oasis	TigerGraph Cloud
Compute unit	Fixed instance tiers (Professional/Enterprise)	Fixed instance tiers on Oasis	Credit-based consumption
Minimum viable footprint	Smallest Professional tier	Smallest Oasis tier	Smallest TigerGraph Cloud tier
Scaling model	Step up to the next fixed tier	Step up to the next fixed tier	Purchase additional credits
Storage cost	Bundled into tier pricing	Bundled into tier pricing	Bundled into tier pricing
Predictability at hundreds of nodes	Medium — tier steps can overshoot	Medium — tier steps can overshoot	Low — credit use is harder to forecast
VEEP fit	Medium	Low	Low

Launch-Scale Monthly Cost — All Six Platforms

A like-for-like estimate for VEEP's launch-scale workload: hundreds of nodes, three regions, and the ingestion pipeline design from Section 06.

Google Cloud Spanner Graph	**≈£280/mo**	Selected technology — full breakdown in Section 05
Amazon Neptune	**≈£455/mo**	Prior AWS-native baseline design
Azure Cosmos DB (Gremlin)	**≈£410/mo**	RU/s minimums push cost up at low request volume
Neo4j AuraDB	**≈£520/mo**	Fixed-tier pricing has no sub-Professional launch tier
ArangoDB Oasis	**≈£340/mo**	Variable — depends on hosting choice and tier
TigerGraph Cloud	**≈£390/mo**	Estimated from an analytics-heavy workload profile

Three-Year Total Cost of Ownership Projection

Projected annual infrastructure spend across VEEP's three scale phases — launch, multi-role expansion, and international rollout (Section 06). Platform cost curve only; engineering cost held constant across all six.

Google Cloud Spanner Graph	£10.1k / 3yr	Scales by adding PUs; no re-platforming cost at any phase
Amazon Neptune	£16.4k / 3yr	Requires instance-resizing decisions at each phase
Azure Cosmos DB (Gremlin)	£14.8k / 3yr	RU/s autoscale reduces but doesn't eliminate step costs
Neo4j AuraDB	£18.7k / 3yr	Each phase is likely to force a tier upgrade
ArangoDB Oasis	£12.2k / 3yr	Lower baseline, but tier steps still apply
TigerGraph Cloud	£14.0k / 3yr	Credit costs grow with traversal-heavy roadmap features

Team Learning Curve & Hiring Market Fit

Fit against VEEP's existing three-person engineering team (Developer, AI Engineer, Data Engineer) and the UK, Canada and Australia hiring markets it recruits from.

Google Cloud Spanner Graph	HIGH FIT	GQL is close to Cypher; team already knows GCP IAM patterns
Amazon Neptune	HIGH FIT	Team already operates AWS-native patterns via Aurora and S3
Azure Cosmos DB (Gremlin)	LOW FIT	No existing Azure footprint; Gremlin unfamiliar to the team
Neo4j AuraDB	MEDIUM FIT	Cypher is the most widely-taught graph language, but a new vendor
ArangoDB	LOW FIT	Smallest hiring pool of the six for AQL specialists
TigerGraph Cloud	LOW FIT	GSQL is a niche skill with a limited regional hiring pool

Ecosystem & Stack Integration Fit

How cleanly each platform fits VEEP's existing Node.js/TypeScript task engine, AlloyDB/Aurora source database, and Claude/GPT-4o AI integration from Section 04.

Google Cloud Spanner Graph	DIRECT FIT	Same VPC as AlloyDB for PostgreSQL; native GCP AI integration path
Amazon Neptune	DIRECT FIT	Same VPC as Aurora PostgreSQL; original design baseline
Azure Cosmos DB (Gremlin)	REQUIRES NEW INFRA	No existing Azure footprint to build from
Neo4j AuraDB	INDIRECT FIT	Runs alongside any cloud, but adds a second vendor and network hop
ArangoDB	INDIRECT FIT	Same integration overhead as AuraDB, on a smaller ecosystem
TigerGraph Cloud	INDIRECT FIT	Analytics-first design adds work for Section 03's transactional patterns

Risk & Vendor Lock-In Assessment

Migration exit-cost and concentration risk for each platform, should VEEP need to change technology after launch.

Google Cloud Spanner Graph	MODERATE RISK	GQL is ISO-standard and portable, though Spanner itself is GCP-only
Amazon Neptune	MODERATE RISK	openCypher/Gremlin patterns are portable; the service is AWS-only
Azure Cosmos DB (Gremlin)	MODERATE RISK	Gremlin is portable; the service itself is Azure-only
Neo4j AuraDB	LOW RISK	The most portable query language; Aura runs on any of the three clouds
ArangoDB	HIGHER RISK	AQL is proprietary, with the smallest ecosystem of the six
TigerGraph Cloud	HIGHER RISK	GSQL is proprietary and analytics-specialised, hardest to re-platform

Weighted Scorecard — Hyperscaler-Native Platforms

Criterion (weight)	Spanner Graph	Neptune	Cosmos DB (Gremlin)
GCP/cloud-native fit (×3)	15	6	6
Managed service maturity (×3)	12	15	12
Query language & standards (×3)	12	15	9
Cost at launch scale (×2)	10	6	6
Hybrid vector support (×2)	10	6	6
WEIGHTED TOTAL (of 65)	59	48	39

Weighted Scorecard — Database-Native Platforms

Criterion (weight)	Neo4j AuraDB	ArangoDB	TigerGraph Cloud
GCP/cloud-native fit (×3)	6	3	3
Managed service maturity (×3)	12	9	9
Query language & standards (×3)	12	6	6
Cost at launch scale (×2)	6	8	6
Hybrid vector support (×2)	8	8	2
WEIGHTED TOTAL (of 65)	44	34	26

Recommendation: Google Cloud Spanner Graph

Across both comparison groups, Google Cloud Spanner Graph scores highest overall (59/65) — driven by its fit with VEEP's three-region residency requirement, its GQL/SQL interoperability, and a launch-scale cost roughly 38% below the next-best hyperscaler option.

WHY GCP WINS

- Highest weighted score of all six platforms evaluated (59 of 65)
- Synchronous multi-region replication matches the trust-score integrity requirement in Section 01, with no manual failover design needed
- GQL is ISO-standard and closely modelled on openCypher, keeping the query patterns in Section 03 largely portable
- Native vector search removes the extra analytics-tier hop that Neptune and Cosmos DB both require for Section 04's hybrid queries
- Lowest launch-scale and three-year cost of the three hyperscaler-native options

TRADE-OFFS ACCEPTED

- Newest of the six platforms in GA (2024), with a smaller production track record than Neptune or Neo4j
- GQL tooling and community resources are less mature than Cypher's, though the gap is closing quickly
- Moves VEEP's original AWS-oriented design onto a new cloud provider
- Requires re-provisioning source infrastructure (AlloyDB for PostgreSQL) alongside the graph layer
- Fewer third-party consultants and pre-built integrations exist today than for Neptune or Neo4j

SOURCE DOCUMENTS *VEEP Workflow Analysis · VEEP Plain-English Report · PHEXARA Brand Strategy Portfolio v2 · VEEP Master Blueprint (Amended)*

What Changes From the Original AWS-Neptune Design

This section's recommendation is now reflected throughout the deck — Sections 05 through 08 describe the Google Cloud Spanner Graph architecture directly. This slide summarises what moved and what stayed the same, for engineering review.

WHAT CHANGES

- Graph database: Amazon Neptune to Google Cloud Spanner Graph
- Source database: Aurora PostgreSQL to AlloyDB for PostgreSQL
- CDC & streaming: DMS/Kinesis to Datastream/Pub/Sub
- Identity, secrets & observability: Cognito/IAM/KMS/CloudWatch to Identity Platform/Cloud IAM/Cloud KMS/Cloud Monitoring
- Query surface: openCypher/Gremlin to GQL + SQL

WHAT STAYS THE SAME

- The 14-node, 20+ relationship ontology from Section 02 — entirely database-agnostic
- The five-phase, seven-month rollout plan from Section 06
- The three-region residency design (UK, Canada, Australia) from Section 01
- The Node.js/TypeScript ingestion-worker language and validation logic from Section 05
- The governance, audit, and RACI model from Section 07

NEXT STEP *Confirm Google Cloud Spanner Graph provisioning against the Phase 1 timeline in Section 06, in place of the original Amazon Neptune provisioning step.*

Technology Comparison: Spanner Graph vs Neo4j Aura vs ArangoDB

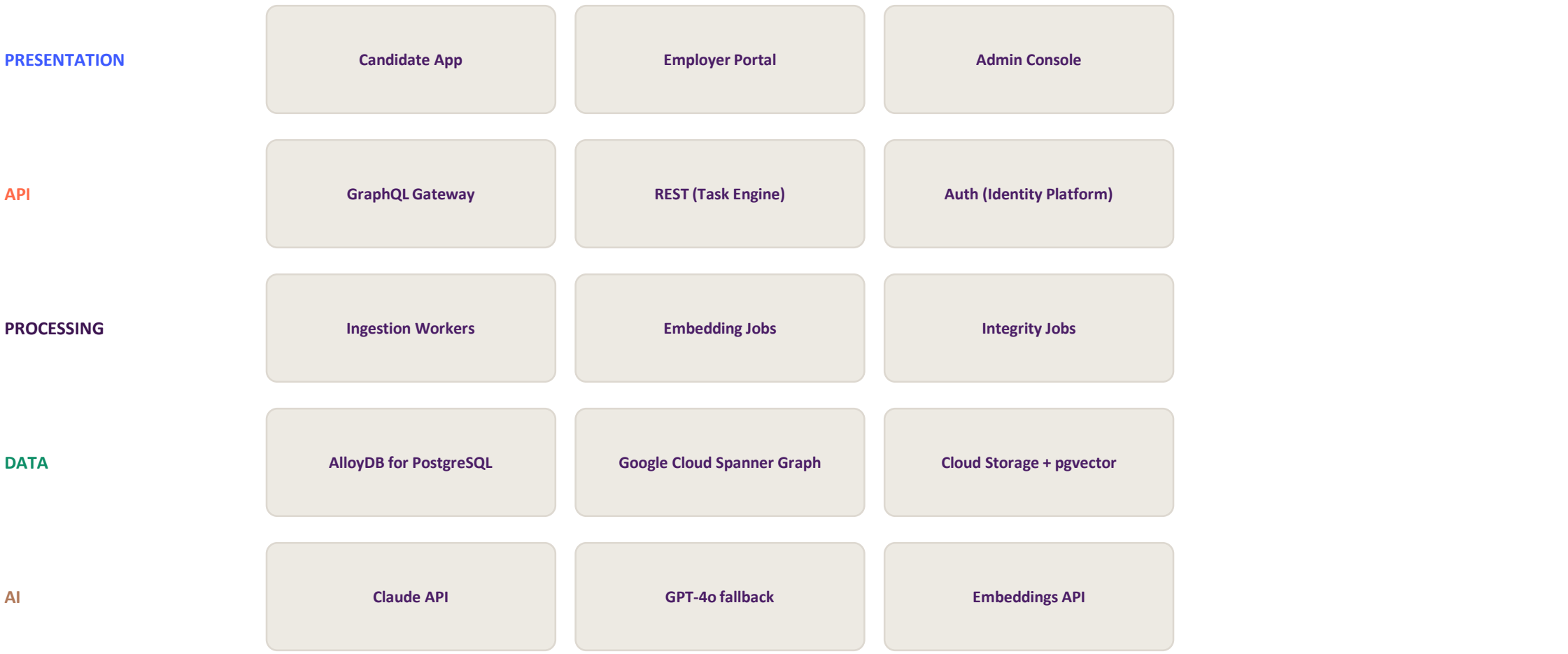
Criterion	Google Cloud Spanner Graph	Neo4j AuraDB	ArangoDB
GCP-native	Native	Third-party, VPC peering required	Third-party, self-hosted or Oasis
Query languages	GQL (ISO Graph Query Language) + SQL	Cypher (native home of Cypher)	AQL (proprietary)
Managed service	Yes — fully managed	Yes — fully managed	Partial — Oasis managed tier
IAM integration	Native Cloud IAM	Separate auth system	Separate auth system
Team familiarity fit	High — same GCP patterns as AlloyDB	Medium — new platform to learn	Low — smallest ecosystem
Launch-scale monthly cost	Lower — pay for provisioned processing units (PUs)	Higher — Aura fixed tiers	Variable — depends on hosting

Selected Technology: Google Cloud Spanner Graph

Google Cloud Spanner Graph (GQL + SQL), deployed in the same VPC as the existing AlloyDB for PostgreSQL cluster.

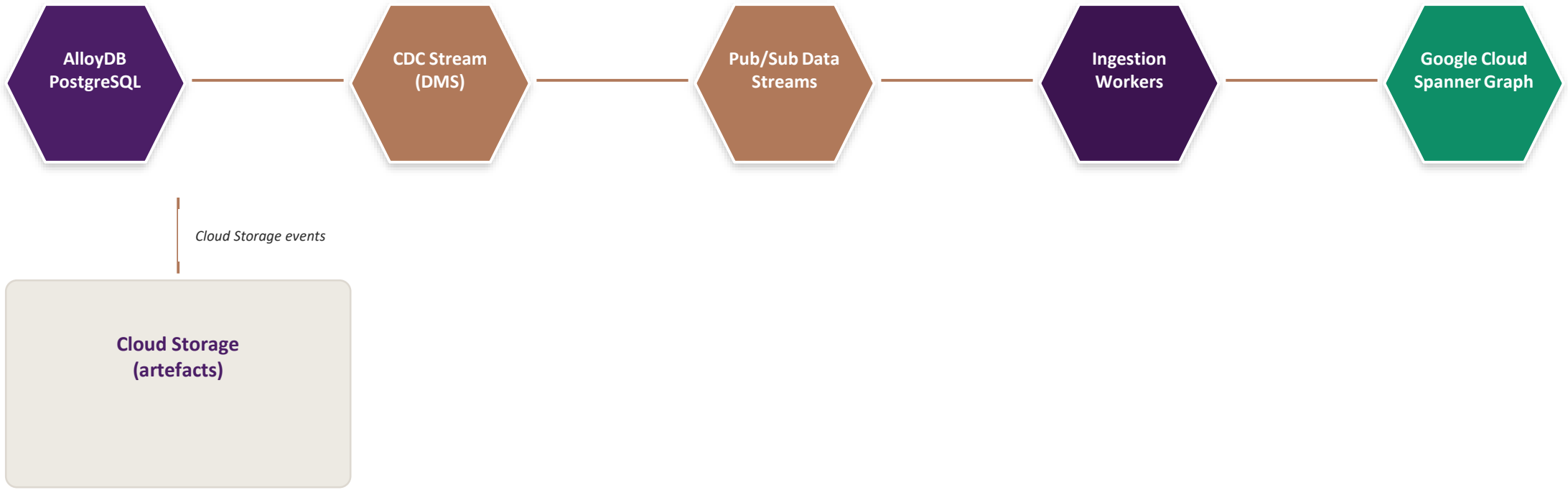
- Native Cloud IAM authentication — reuses the security model already governing AlloyDB and Cloud Storage
- GQL is syntactically modelled on openCypher, so the query patterns in Section 03 translate with minimal rework
- Fully managed — no graph database administration overhead for a three-person engineering team
- Provisioned-capacity pricing keeps launch-scale monthly cost predictable and low, per the KPI in Section 01
- Sits inside the same VPC as AlloyDB, so CDC ingestion in the following slides has no public network hop

Target-State Architecture: Full System Diagram



All layers deployed inside one VPC across three GCP regions (UK, Canada, Australia) — detailed in the network diagram ahead.

Data Flow: Source Systems to Knowledge Graph



Every write is captured as a change event at source — the graph is never written to directly by application code, only by this pipeline.

Change Data Capture Pipeline — AlloyDB to Spanner Graph

Google Cloud Datastream runs in ongoing-replication mode, reading the AlloyDB for PostgreSQL write-ahead log and emitting one change event per row modification.

1	**Source capture**	DMS reads the AlloyDB for PostgreSQL logical replication slot
2	**Event shaping**	Row change is mapped to a node or edge mutation using the ontology from Section 02
3	**Ordering guarantee**	Pub/Sub partition key = candidate_id, guaranteeing in-order delivery per candidate
4	**Idempotent write**	Ingestion worker uses MERGE semantics — replaying an event never creates a duplicate
5	**Dead-letter handling**	Malformed events route to a DLQ and page the on-call Data Engineer

Event Streaming Layer — Google Cloud Pub/Sub

Pub/Sub sits between DMS and the ingestion workers, decoupling capture rate from write rate and giving the pipeline replay capability.

Shard count	4 shards at launch, auto-scaling enabled
Retention	24 hours standard, extendable to 7 days for incident replay
Partition key	candidate_id — preserves per-candidate event order
Consumer	Ingestion Workers (Cloud Run functions, batch size 100)
Failure isolation	Per-shard failure does not block other shards' consumers

Ingestion Service Design — Node.js Workers

Stateless Cloud Run function instances, written in the same Node.js/TypeScript stack as the existing Task Engine, so the Developer and Data Engineer share one language across the platform.

INGESTION WORKER — CORE LOGIC (SIMPLIFIED)

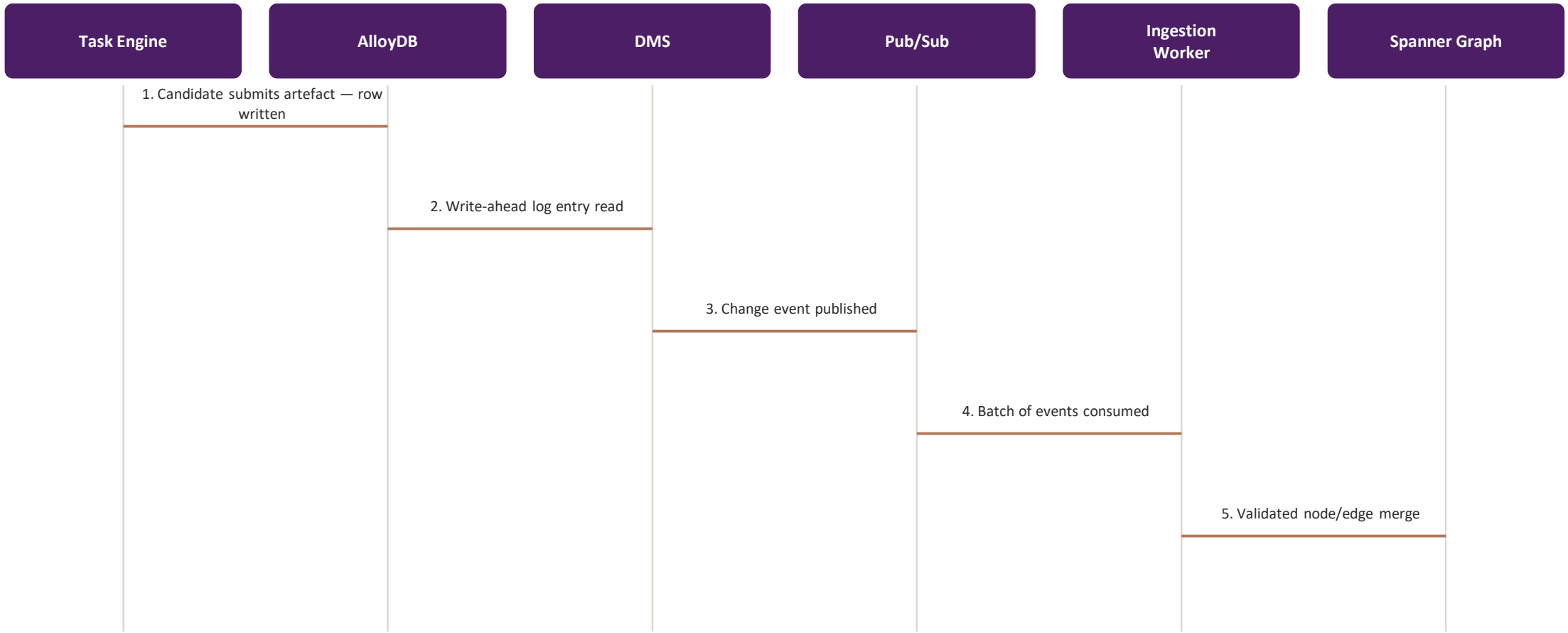
```
export async function handleEvent(event: ChangeEvent) {
  const mapping = ONTOLOGY_MAP[event.table];
  if (!mapping) return skip(event);

  const mutation = mapping.toGraphMutation(event);
  await validateAgainstOntology(mutation);      // Section 02 schema
  await validateDataQualityRules(mutation);      // Section 04 rules

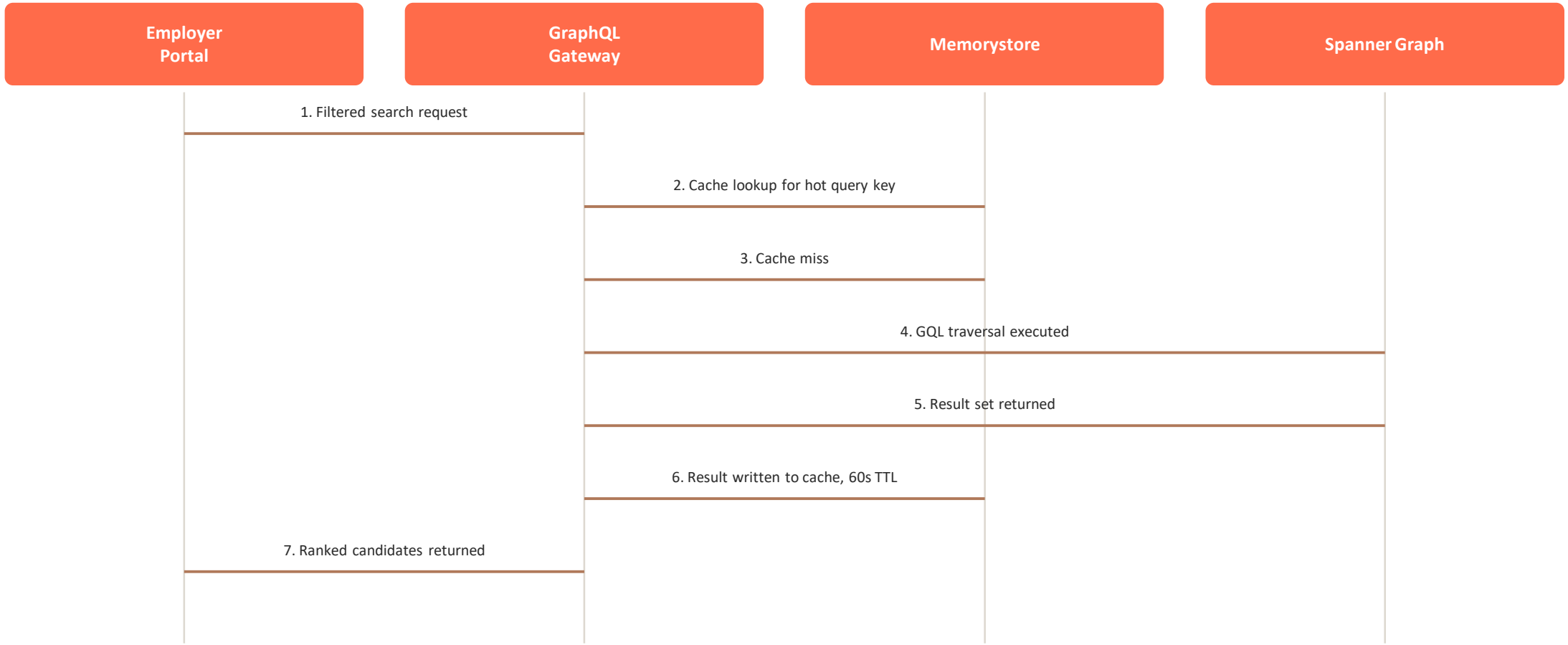
  await spanner.mergeNode(mutation.node, { idempotencyKey: event.sourceEventId });
  await spanner.mergeEdges(mutation.edges);

  await metrics.recordWriteLag(event.committedAt); // Section 01 KPI
}
```

Graph Write Path — Sequence Diagram



Graph Read Path: Query API Sequence Diagram



API Layer — GraphQL Gateway over Spanner Graph

A single GraphQL schema exposes graph traversals as typed queries, so front-end teams never write GQL directly.

GRAPHQL SCHEMA EXCERPT

```
type VerifiedProfile {
  id: ID!
  candidate: Candidate!
  trustScore: TrustScore!
  scores: [Score!]!
  evidenceChain: [Artefact!]!      # resolver runs Query 2 from Section 03
}

type Query {
  searchCandidates(
    region: Region, role: String,
    minTrustScore: Float, competencies: [String!]
  ): [VerifiedProfile!]!            # resolver runs Query 1 from Section 03
}
```

Caching Strategy — Memorystore for Hot Traversals

The existing Redis Memorystore cluster gains a second logical database for graph query results, separate from the Task Engine's BullMQ queues.

Cache key	Hash of (query name + parameters), e.g. searchCandidates:region=UK:role=...:minTrust=80
TTL	60 seconds for discovery queries; 5 seconds for evidence-chain queries (Section 03, Query 2)
Invalidation	Explicit invalidation on any write affecting the candidate_id in a cached key
Never cached	Fraud/tamper detection traversals (Section 03) always run live, uncached

Security Architecture — IAM, VPC, Encryption

IDENTITY & ACCESS

- Cloud IAM database authentication — no static credentials
- Per-service IAM roles, least privilege, no shared service accounts
- Identity Platform-issued JWTs validated at the GraphQL Gateway

NETWORK

- Spanner Graph reachable only from within the VPC — no public endpoint
- Private subnets for all data-layer services
- VPC endpoints for Cloud Storage and Pub/Sub — no traffic over the public internet

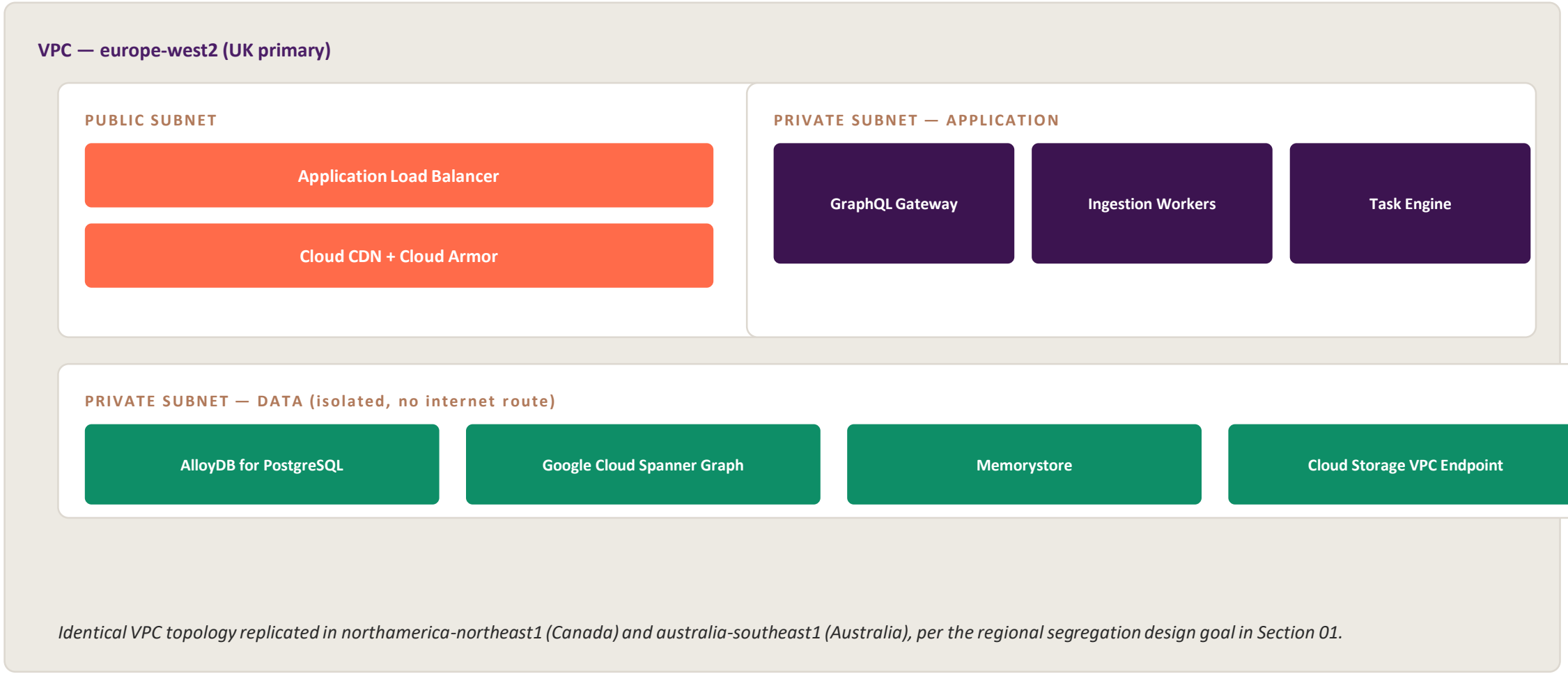
ENCRYPTION

- Encryption at rest via Cloud Cloud KMS, customer-managed key
- TLS 1.2+ for all in-transit traffic, including CDC pipeline
- Artefact hashes (SHA-256) independently verifiable outside the graph

Access Control Model — Role-Based Graph Permissions

SERVICE / ACTOR	PERMISSION	KEY RESTRICTION
Candidate App	Read own Candidate, Artefact, Score, VerifiedProfile subgraph only	*Cannot read other candidates' data*
Employer Portal	Read VerifiedProfile, TrustScore, Score for discovered candidates	*Cannot read raw Artefact content or AIPersona internals*
Admin Console	Read all nodes; write restricted to governance-approved operations	*All writes logged per Section 07 audit requirements*
Ingestion Worker	Write nodes/edges per the ontology mapping only	*Cannot delete evidence nodes under any condition*
Integrity Job	Read all; write only to a dedicated IntegrityFlag node type	*Cannot modify Score or TrustScore values*

Network Architecture Diagram



Identical VPC topology replicated in northamerica-northeast1 (Canada) and australia-southeast1 (Australia), per the regional segregation design goal in Section 01.

Backup, Snapshot & Disaster Recovery Strategy

Automated snapshots

Spanner Graph continuous backup to Cloud Storage, 35-day retention

Point-in-time recovery

Restore to any second within the retention window

Cross-region replication

Snapshots replicated to a secondary region nightly

Recovery Time Objective

4 hours for full graph restoration from snapshot

Recovery Point Objective

5 minutes — bounded by the CDC pipeline's write-lag KPI

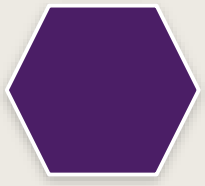
DR drill cadence

Quarterly, restoring into an isolated test cluster

High Availability & Multi-Region Design

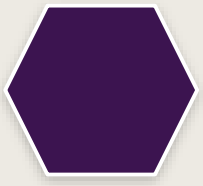
Spanner Graph synchronously replicates every write across zones using Paxos consensus, so there is no writer/reader promotion step and no replication lag to manage.

Availability Zone A



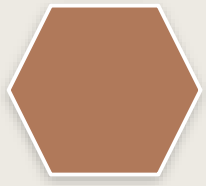
Spanner Graph Voting Replica
(zone A)

Availability Zone B



Spanner Graph Voting Replica
(zone B)

Availability Zone C



Spanner Graph Voting Replica
(zone C)

Zero-downtime failure handling: if a zone is lost, the remaining voting replicas continue serving reads and writes immediately under Paxos majority quorum — no promotion step, no failover window.

Observability Stack — Metrics, Logs, Traces

METRICS

- Google Cloud Monitoring — Spanner-native metrics (CPU, IOPS, query latency)
- Custom metrics: write lag, cache hit rate, evidence-chain completeness

LOGS

- Spanner Graph audit log, streamed to Cloud Logging
- Ingestion worker structured logs, one line per event processed

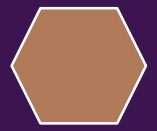
TRACES

- Google Cloud Trace across GraphQL Gateway → Spanner Graph → cache
- End-to-end trace from AlloyDB write to graph-queryable, for lag debugging

Cost Model & Capacity Planning

COST LINE	ESTIMATE
Spanner Graph (100 processing units, regional instance)	**≈ £140 / month**
Pub/Sub (4 topics, launch-scale throughput)	**≈ £25 / month**
Datastream replication (AlloyDB CDC)	**≈ £50 / month**
Cloud Run functions ingestion workers (est. volume)	**≈ £20 / month**
Memorystore incremental usage (shared instance)	**≈ £15 / month**
Cloud Monitoring, Cloud Trace, backups	**≈ £25 / month**

Total estimate: ≈ £280 / month at launch scale — within the < £600 KPI set in Section 01, and roughly 38% below the equivalent AWS/Neptune estimate.



Build & Deployment Plan

Environments, CI/CD, backfill, and a five-phase rollout mapped directly to VEEP's existing 7-month build plan.

Deployment Environments

Development

DATA

Shared Spanner Graph instance, synthetic data only, reset weekly

USE

Feature branches, local GraphQL Gateway

Staging

DATA

Full-scale Spanner Graph instance, anonymised production-shaped data

USE

Pre-release validation, load testing

Production

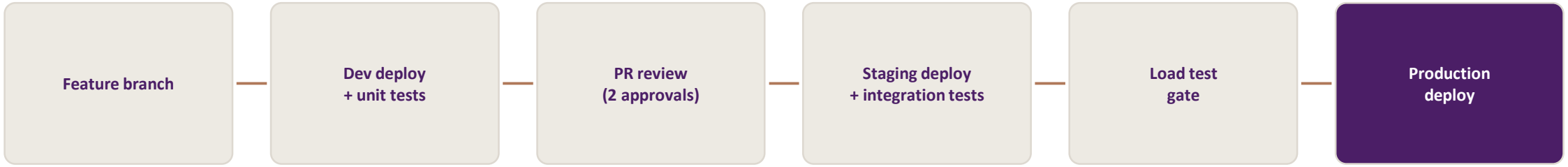
DATA

Multi-AZ Spanner Graph instance per region, real candidate and employer data

USE

Live traffic, strict change control

Environment Promotion Workflow



SCHEMA CHANGES GET AN ADDITIONAL GATE

Any change to the ontology in Section 02 additionally requires sign-off from the Schema Governance Board (Section 07) before it can be merged — application code changes do not.

CI/CD Pipeline for Graph Schema Changes

GITHUB ACTIONS — SCHEMA-CHANGE WORKFLOW

```
on: pull_request, paths: ['ontology/**']

jobs:
  validate-ontology:
    - Lint schema definition against Section 02 property standards
    - Check: additive only — no renamed or removed node/edge types
    - Diff against production schema, output human-readable change summary

  test-migration:
    - Apply migration to an ephemeral Spanner Graph instance
    - Run full sample-query regression suite (Section 03 queries 1–3)
    - Assert p95 latency KPI (Section 01) unaffected

  require-approval:
    - Schema Governance Board sign-off required (Section 07)
    - Merge blocked until approval label applied
```

Schema Migration & Versioning Strategy

The ontology in Section 02 is versioned independently of application code, so a schema version can be referenced from any node's _source_system metadata.

Additive changes (new node type, new edge type, new optional property)

Deployed directly, no migration required, versioned as a minor bump

Required-property changes

Deployed as a backfill job first, property becomes required only once 100% populated

Deprecations

Old property or edge type marked deprecated, dual-written for one full quarter, then removed

Breaking changes

Never applied directly — always modelled as an additive change plus a phased migration

Initial Data Backfill Plan

Before the CDC pipeline goes live, existing AlloyDB for PostgreSQL history must be loaded into Spanner Graph as a one-time historical backfill.

1	**Snapshot export**	Point-in-time export of all AlloyDB tables relevant to the ontology, to Cloud Storage
2	**Batch transform**	Spark job on Google Cloud Dataflow maps each row to its node/edge representation, per Section 02
3	**Bulk load**	Spanner Graph bulk loader ingests transformed data directly from Cloud Storage — far faster than row-by-row writes
4	**Validation**	Row counts and spot-checked traversals compared against source, per the next slide
5	**Cutover**	CDC pipeline switched on, resuming from the exact snapshot timestamp — no gap, no duplication

Backfill Validation & Reconciliation

Row-count parity	Node count in Spanner Graph == row count in source AlloyDB table, per table
Edge-count parity	Foreign-key relationship count == edge count, per relationship type
Sample traversal match	100 randomly sampled candidates: run Query 2 (Section 03), compare to manual AlloyDB join
Hash integrity spot-check	1,000 randomly sampled artefacts: recompute SHA-256, compare to stored VerificationHash
Sign-off	Data Engineer and Cybersecurity Analyst jointly sign off before cutover proceeds

Phased Rollout Plan — Overview

Five phases, aligned one-to-one with VEEP's existing 7-month build plan from the Workflow Analysis — the graph is never a separate track running out of sync with product delivery.

1. **Cybersecurity Analyst Track**
   Months 1–2
2. **Employer Portal Graph Queries**
   Month 3
3. **Multi-Role Expansion**
   Months 4–5
4. **Public Launch Scale-Out**
   Month 6+
5. **International Expansion**
   Year 2–3

Cybersecurity Analyst Track

MONTHS 1–2

GOAL

Stand up the graph for a single role track, proving the full evidence chain end to end before expanding.

IN SCOPE FOR THIS PHASE

- Spanner Graph instance provisioned in UK region only
- Ontology limited to nodes touched by the Cybersecurity Analyst track
- CDC pipeline live for this role's tasks and artefacts
- Sample queries 1–3 (Section 03) validated against real data

PRIMARY OWNER

Data Engineer, with AI Engineer supporting evaluator grounding (Section 04)

Employer Portal Graph Queries

MONTH 3

GOAL

Expose the discovery traversal to real employer-facing search, replacing any interim relational filter.

IN SCOPE FOR THIS PHASE

- GraphQL Gateway deployed, employer portal migrated to it
- Caching strategy (Section 05) enabled for discovery queries
- Explainability panel (Section 04) shipped in the employer portal
- Load testing against expected employer traffic

PRIMARY OWNER

Developer, with Data Engineer owning the GraphQL resolver layer

Multi-Role Expansion

MONTHS 4–5

GOAL

Extend the ontology and ingestion pipeline to cover the remaining 14 role tracks.

IN SCOPE FOR THIS PHASE

- Role and Competency nodes populated for all 15 role tracks
- Ingestion mapping extended, following the additive-schema principle
- Fraud/tamper detection job (Section 03) running continuously
- Recommendation engine (Section 04) trained on early placement data

PRIMARY OWNER

Data Engineer and AI Engineer jointly

Public Launch Scale-Out

MONTH 6+

GOAL

Scale infrastructure for public candidate volume and enable multi-AZ high availability.

IN SCOPE FOR THIS PHASE

- Spanner Graph reader replicas added per the HA design in Section 05
- Full observability stack (Section 05) live with on-call alerting
- Disaster recovery drill completed before public launch date
- Cost model reviewed against actual usage, adjusted if needed

PRIMARY OWNER

Developer, with Cybersecurity Analyst validating security posture

International Expansion

YEAR 2–3

GOAL

Replicate the regional VPC topology for Canada and Australia, per the network design in Section 05.

IN SCOPE FOR THIS PHASE

- northamerica-northeast1 and australia-southeast1 clusters provisioned
- Region-partitioned data confirmed structurally isolated (Section 01 principle)
- Local compliance regime properties populated on Region nodes
- Cross-region query capability evaluated for future global employer search

PRIMARY OWNER

Founders and technical co-founders, informed by regional compliance review

Testing Strategy — Unit, Integration, Graph Consistency

Unit tests	Ingestion mapping functions — one source table row in, one correct node/edge mutation out
Integration tests	Full pipeline, AlloyDB write to Spanner Graph-queryable, against an ephemeral test cluster
Graph consistency tests	Run all sample queries (Section 03) against known fixture data, assert exact results
Contract tests	GraphQL schema (Section 05) checked for breaking changes on every pull request

Load & Performance Testing Plan

Target load	5x projected launch-month traffic, sustained for 30 minutes
Tooling	k6, scripted against the GraphQL Gateway, run from CI on every release candidate
Pass criteria	p95 latency KPIs from Section 01 held throughout the test window
Write-path test	Simulated CDC burst of 10x normal event volume, write lag measured against KPI
Failure injection	One reader replica killed mid-test, confirms HA failover per Section 05

Rollback & Contingency Plan

IF A SCHEMA DEPLOYMENT FAILS

- Additive-only design means rollback is simply halting ingestion of the new node/edge type
- No destructive rollback is ever required, by construction (Section 06, previous slides)
- Ingestion worker redeployed to prior version, resumes from last successful checkpoint

IF SPANNER GRAPH ITSELF IS UNAVAILABLE

- GraphQL Gateway falls back to a cached, degraded read-only mode using last-known Memorystore results
- Employer discovery shows a 'results may be delayed' notice rather than failing outright
- AlloyDB for PostgreSQL remains fully operational throughout — the system of record is never at risk

Cutover Runbook — Step by Step

1. Freeze schema changes 48 hours before cutover window
2. Take final AlloyDB snapshot; begin bulk load into Spanner Graph per Section 06 backfill plan
3. Run full reconciliation suite; require sign-off from Data Engineer and Cybersecurity Analyst
4. Enable CDC pipeline in shadow mode — writing, but not yet serving reads
5. Run smoke tests: Section 03 sample queries, against live shadow data
6. Switch GraphQL Gateway to serve from Spanner Graph; monitor error rate and latency for 1 hour
7. Declare cutover complete; resume normal schema-change cadence

Data Governance Framework

Ownership	Every node type has one named owning team; the ontology (Section 02) records this in its documentation
Change control	Schema Governance Board (Section 07) approves all additive changes; no unilateral edits
Data retention	Evidence nodes retained indefinitely; PII-bearing properties reviewed under regional policy
Access review	Quarterly review of the role-based permissions set out in Section 05

Regional Compliance — UK ICO / GDPR Data Residency

The graph's regional partitioning (Section 01 principle, Section 05 network design) is what makes UK GDPR compliance structural rather than procedural.

Data residency	UK candidate and employer data held exclusively in the europe-west2 Spanner Graph instance
Right to erasure	Candidate deletion request triggers a governed process — see next slide for regional nuance
Right to explanation	The explainability panel (Section 04) directly supports Article 22 automated-decision transparency
email_hash property	Candidate email is stored hashed in the graph (Section 02); the reversible identity lives only in AlloyDB, access-controlled separately

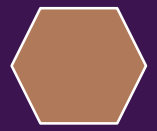
Regional Compliance — Canada & Australia

CANADA — PIPEDA

- Data held in northamerica-northeast1 Spanner Graph instance (Phase 5, Section 06)
- Consent model for candidate data reuse mirrors the UK approach
- Breach notification workflow reuses the incident response playbook (Section 07)

AUSTRALIA — PRIVACY ACT

- Data held in australia-southeast1 Spanner Graph instance (Phase 5, Section 06)
- Australian Privacy Principles mapped against the access control model (Section 05)
- Regional compliance_regime property on the Region node drives automated policy checks



Operations & Governance

How the graph is owned, monitored, and kept trustworthy once it is live in production.

Operational Ownership: RACI for the Knowledge Graph

ACTIVITY	Founders	Developer	Data Engineer	AI Engineer	Cybersecurity Analyst
Ontology changes	I	C	R	C	C
Ingestion pipeline	I	R	A	C	C
Query API & caching	I	R	C	C	I
Security & access	A	C	C	I	R
Incident response	I	R	R	C	A
Cost management	A	C	R	I	I

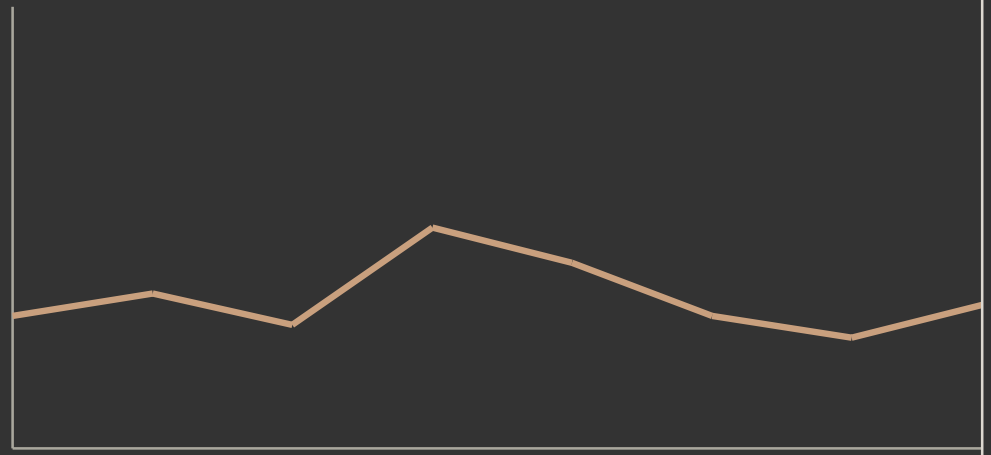
R = Responsible A = Accountable C = Consulted I = Informed

Monitoring Dashboard — Design Mock-Up

VEEP KNOWLEDGE GRAPH — OPERATIONS DASHBOARD

2.1s	340ms	78%	0
Write lag	p95 query latency	Cache hit rate	Open integrity flags

QUERY LATENCY, TRAILING 24H



RECENT INTEGRITY JOB RUNS

04:00 UTC	Fraud/tamper scan	0 flags
03:00 UTC	Hash integrity check	0 flags
02:00 UTC	Evaluator anomaly scan	1 flag — reviewed

Alerting Thresholds & Escalation Paths

CONDITION	SEVERITY	ACTION
`Write lag > 30s`	P1	Page on-call Data Engineer immediately
`p95 query latency > 1.5s for 5 min`	P1	Page on-call Developer immediately
`Any fraud/tamper flag raised`	P0	Page on-call Cybersecurity Analyst immediately
`Cache hit rate < 40% for 15 min`	P2	Ticket created, reviewed next business day
`Monthly cost > 120% of estimate`	P2	Notify Founders in weekly digest

Graph Health Metrics

Node growth rate	New nodes per day, by type — tracks adoption against roadmap projections
Edge-to-node ratio	Healthy range 2.5–4.0 per candidate; sudden drops suggest an ingestion gap
Orphan node rate	Nodes with zero edges — should trend to zero; any persistent orphans investigated
Query latency percentiles	p50, p95, p99 tracked separately per query type, against Section 01 KPIs
Evidence-chain completeness	% of TrustScore nodes with a fully resolvable path to VerificationHash — target 100%

Data Quality Monitoring & Remediation Workflow



Because the CDC pipeline is replayable (Section 05), remediation never requires a manual data-patching script against the graph directly.

Audit Logging & Compliance Reporting

Spanner Graph's audit log, combined with the ingestion worker's structured logs, gives a complete record of every read and write against the graph.

What is logged	Every query (hashed for PII), every write, actor identity, timestamp, source IP
Retention	12 months in Cloud Logging, archived to Cloud Storage Archive thereafter
Compliance reports	Monthly automated report: access by role, evidence-chain completeness, flagged anomalies
Employer disclosure requests	Full audit trail can be generated per candidate on request, supporting Section 06's Article 22 commitment

Incident Response Playbook

1	**Detect**	Alerting (Section 07) fires, or a fraud/tamper flag is raised by an integrity job
2	**Contain**	Affected node/edge type write-blocked at the ingestion worker if integrity is in question
3	**Investigate**	Full audit trail (previous slide) and provenance traversal (Section 03) reconstruct what happened
4	**Remediate**	Fix applied via the replay mechanism (Section 07); no direct graph edits
5	**Report**	Incident summary logged; Cybersecurity Analyst assesses disclosure obligations

Change Management & Schema Governance Board

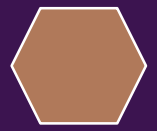
A lightweight board — not a bureaucracy — that exists solely to protect the additive-schema principle from Section 01 as the team grows.

MEMBERS

- Data Engineer (chair)
- Developer
- AI Engineer, for changes touching evaluation or persona context
- Cybersecurity Analyst, for changes touching evidence or access control

REMIT

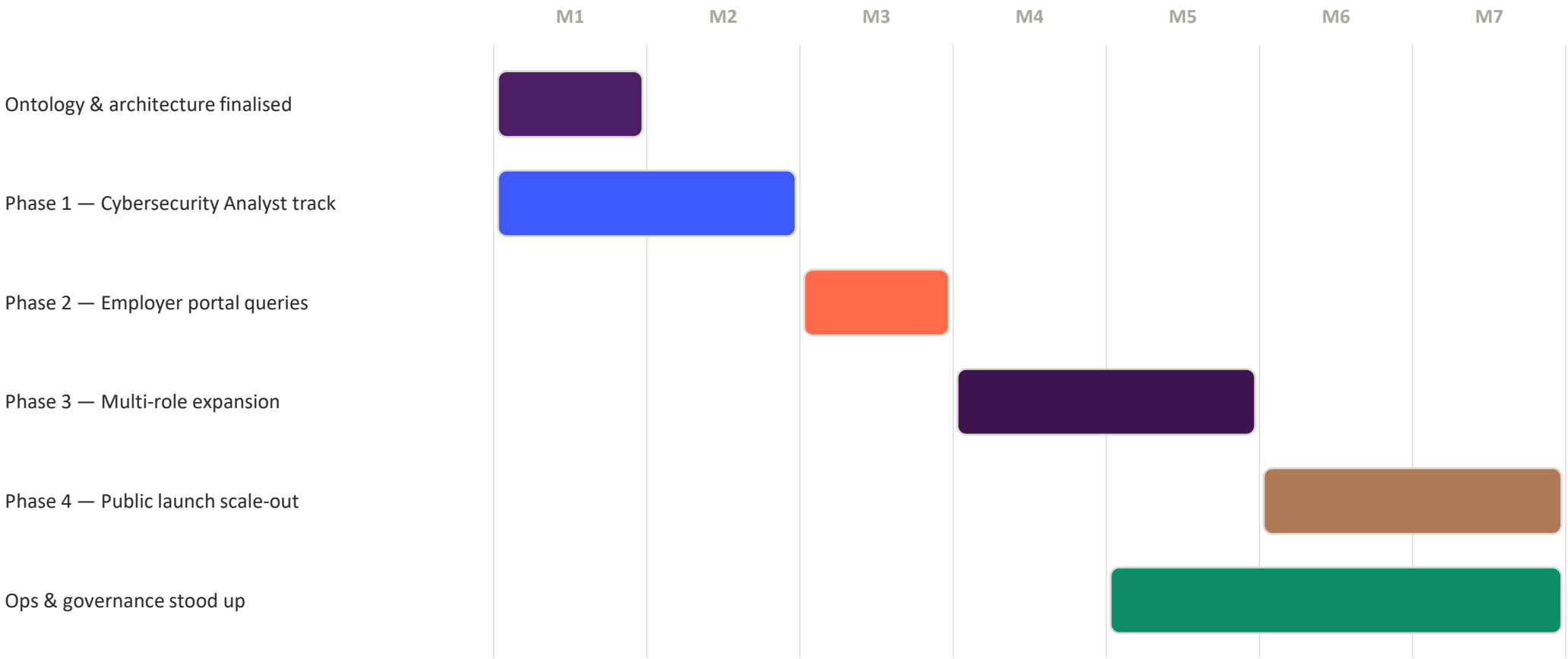
- Approve or reject proposed ontology changes (Section 06 CI/CD gate)
- Maintain the published schema documentation as the single source of truth
- Review the quarterly access audit (Section 06 governance framework)
- Meets asynchronously — no standing meeting required for launch-scale volume



Roadmap, Team & Investment

Timeline, ownership, risk, and the budget required to build and run the knowledge graph.

Programme Roadmap — 7-Month Timeline



Team & Ownership Model

Data Engineer	Ontology, ingestion pipeline, backfill, graph health metrics
Developer	GraphQL Gateway, CI/CD, caching, employer/candidate app integration
AI Engineer	Hybrid retrieval, graph-grounded evaluation, persona context assembly
Cybersecurity Analyst	Security architecture, access control, incident response, audits
Founders / Technical Co-Founders	Programme sponsorship, regional expansion decisions, budget

Risk Register — Top Risks & Mitigations

RISK	SEVERITY	MITIGATION
CDC pipeline write lag exceeds KPI under real load	Medium	Load testing (Section 06) before each phase; auto-scaling Pub/Sub shards
Schema drift between AlloyDB and graph ontology	Medium	CI/CD schema validation gate (Section 06); Schema Governance Board review
Small team stretched across five workstreams	High	Phased rollout (Section 06) limits concurrent scope per phase
Regional compliance requirement discovered late	Low	Region node design (Section 02) built for compliance flexibility from day one
Cost overrun at scale	Low	Monthly cost KPI (Section 01) monitored via dashboard (Section 07)

Budget & Cost Summary

≈ £280/mo	7 months	0	5
Infrastructure at launch scale	**To full public launch scale-out**	**New headcount required**	**Phases, each independently valuable**
(Section 05)	*(Section 06)*	*existing team covers all roles*	*no single big-bang delivery*

The knowledge graph programme is designed to be funded from existing engineering capacity and existing GCP spend headroom. It requires no new hire, no new vendor contract, and no infrastructure outside the GCP project VEEP already operates in.

CLOSE

Summary — Key Takeaways

1. VEEP's trust promise depends on provable, multi-hop evidence chains — a knowledge graph is the correct data structure for that job, not an add-on for its own sake

2. The ontology is fully specified: 14 node types, 20+ relationship types, all grounded in VEEP's existing candidate-to-placement journey

3. Google Cloud Spanner Graph, deployed inside VEEP's existing GCP VPC, meets every evaluation criterion at an estimated £280/month at launch scale, roughly 38% below the equivalent AWS/Neptune estimate

4. The five-phase rollout maps directly onto VEEP's existing 7-month build plan — this is additive work, not a competing roadmap

5. Governance, monitoring, and access control are designed in from the start, not retrofitted after a security incident

Thank You

VEEP Knowledge Graph — Design & Deployment

NEXT STEPS

1. Engineering leadership review and sign-off on the ontology (Section 02)
2. Confirm Google Cloud Spanner Graph provisioning against Phase 1 timeline (Section 06)
3. Schedule the first Schema Governance Board session (Section 07)